

Associate Architect 양성과정

DESIGN PATTERNS

2023. 11.

중앙대학교 소프트웨어학부

이찬근



CONTENTS

- 01 Command Pattern**
- 02 Chain-of-Responsibility Pattern**
- 03 Prototype Pattern**
- 04 Proxy Pattern**
- 05 Visitor Patterns**
- 06 Facade Patterns**

A vertical blue sidebar on the left side of the slide. It contains several decorative elements: a large blue circle at the top left with a white dotted pattern; a smaller blue circle at the top right with a white diagonal line pattern; a vertical white line with a small white circle in the middle and three dots at the bottom; a large blue circle at the bottom left with a white dotted pattern; a smaller blue circle at the bottom right with a white diagonal line pattern; and a white curved shape at the bottom left with a small red circle and three dots.

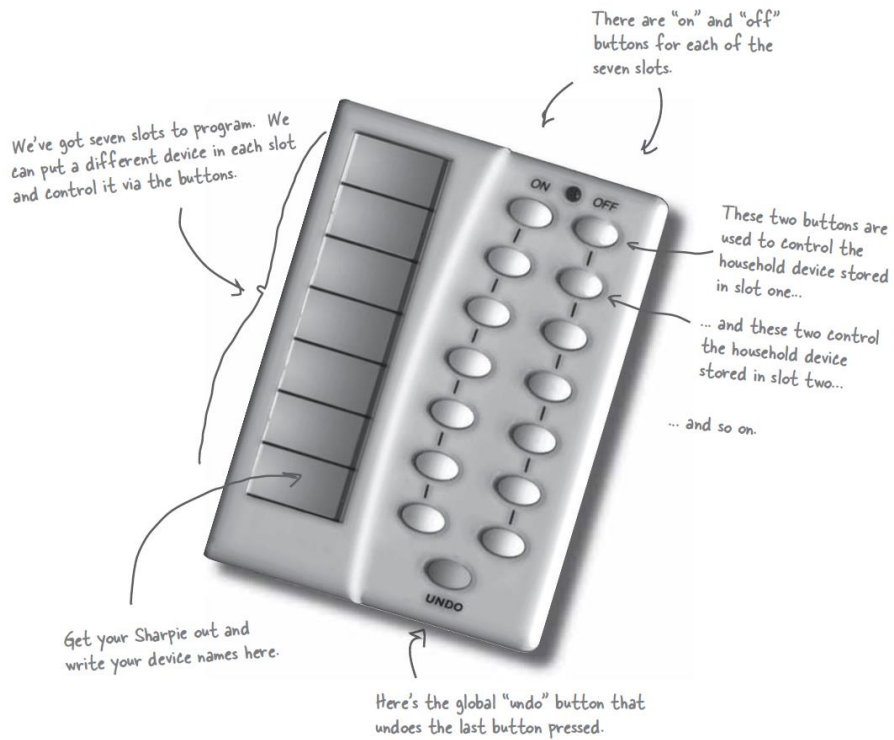
01

Command Pattern

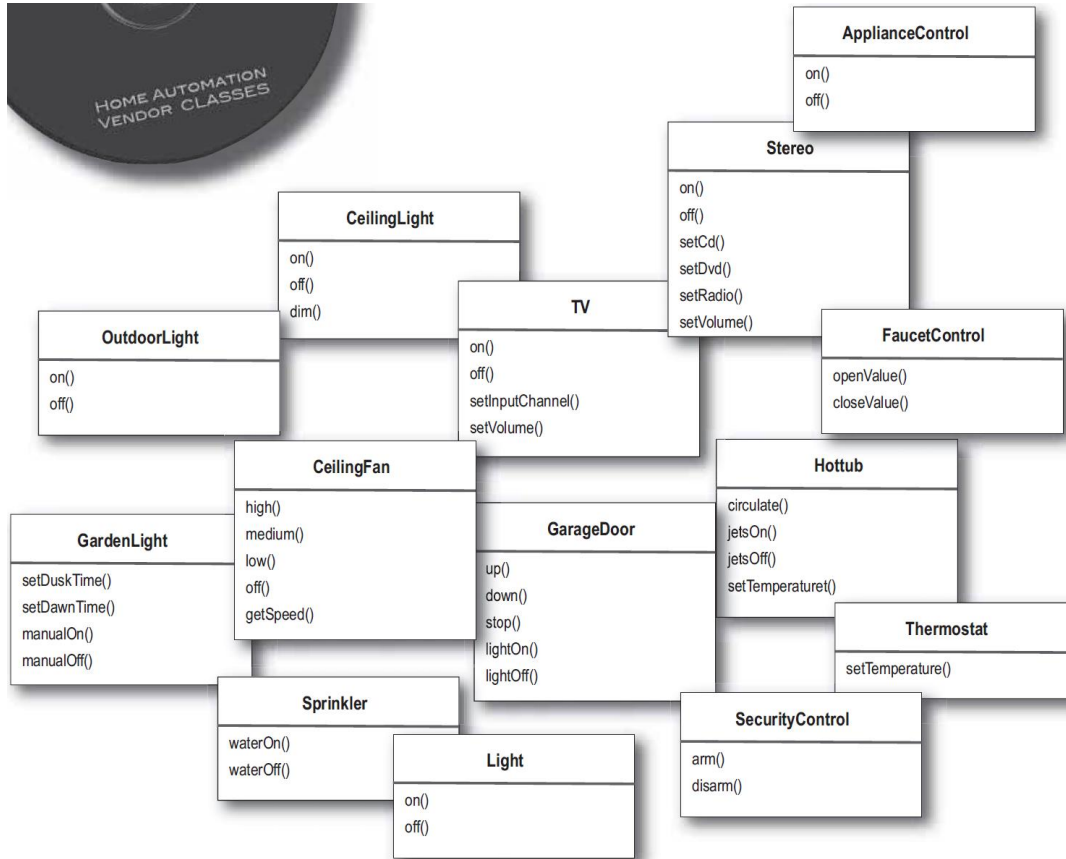
Command Pattern

- Purpose
 - Encapsulates a request as an object.
 - This allows the request to be handled in traditionally **object based relationships** such as queuing and callbacks.
- Use When
 - Requests need to be specified, queued, and executed at variant times or in variant orders.
 - A history of requests is needed.
 - The invoker should be decoupled from the object handling the invocation.

Remote Controller



Our Vendor Classes



Without Using Command Pattern

- Never do this!

```
if(command == Slot1On)
    light.on();

else if(command == Slot1Off)
    light.off();
```

- Every time you want to modify the behavior of a button, you need to touch the client code!

Implementing Command interface

- Command interface

```
public interface Command {  
    public void execute();  
}
```

- You may want to include `undo()` for undo operation later.

Implementing a command

```
public class LightOnCommand implements Command {  
    Light light; // Stores info. about its receiver  
  
    public LightOnCommand(Light light){  
        this.light = light;  
    }  
    public void execute() {  
        light.on();  
    }  
}
```

Using the command object (Building Invoker)

```
public class SimpleRemoteControl{
    Command slot;

    public simpleRemoteControl() { }

    public void setcommand( Command command ) {
        slot = command;
    }

    public void buttonwaspressed() {
        slot.execute();
    }
}
```

```
public class RemoteControlTest {  
    public static void main(String[] args) {  
        SimpleRemoteControl remote = new SimpleRemoteControl();  
  
        Light light = new light();  
  
        LightOnCommand lightOn = new LightOnCommand( light );  
  
        remote.setCommand( lightOn );  
  
        remote.buttonWasPressed();  
    }  
}
```

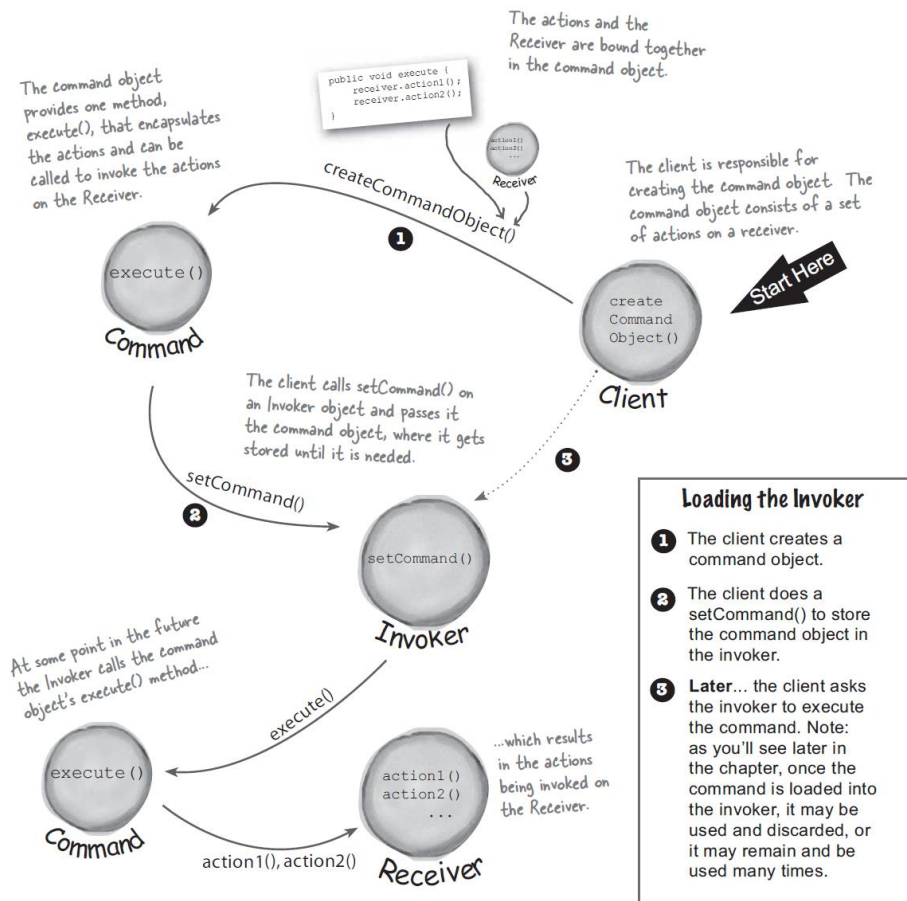
Command Pattern

- LightOn, LightOff, TVOn, TVOff, etc.

```
public class LightOffCommand implements Command {  
    Light light; // Stores info. about its receiver  
  
    public LightOffCommand(Light light) {  
        this.light = light;  
    }  
    public void execute() {  
        light.off();  
    }  
}
```

- Command Pattern encapsulates a request as an object:
 - Each command object exposes only execute (and undo) method

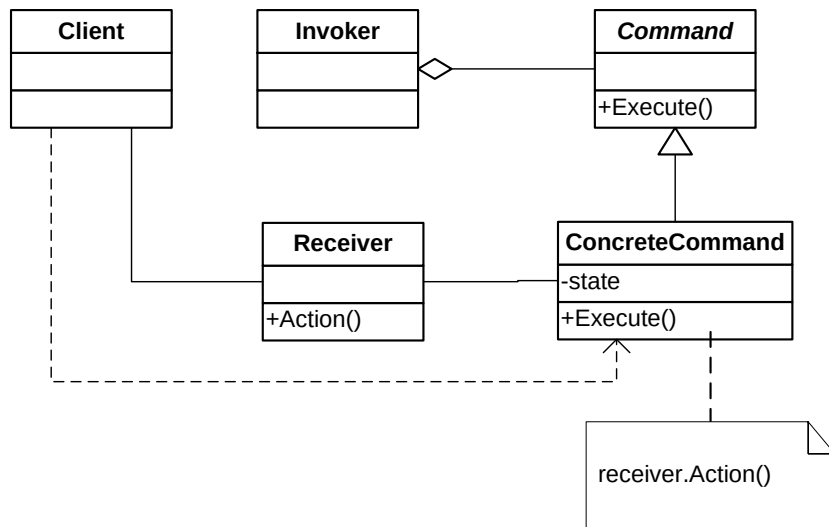
Our Approach



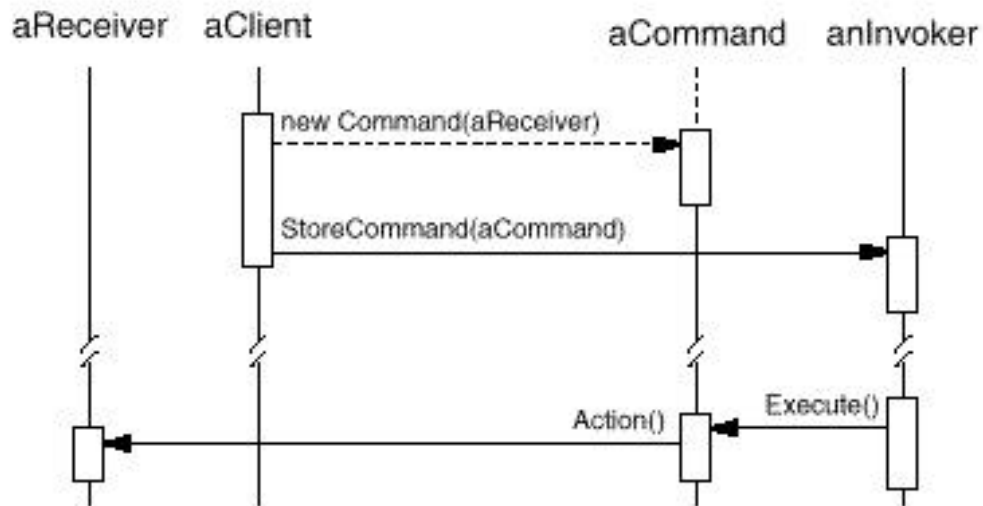
Concepts of Command Pattern

Encapsulates a request as an object

thus, let you parameterize clients with different requests, queue or log requests, and support undoable operations



Collaborations



Extending the Remote Control

```
public class RemoteControl {  
  
    Command[] oncommands;  
    Command[] offcommands;  
  
    RemoteControl() {  
        oncommands = new Command[7];  
        offcommands = new Command[7];  
  
        Command noCommand = new NoCommand();  
  
        for(int i = 0; i < 7; i++) {  
            onCommand[i] = noCommand;  
            offCommand[i] = noCommand;  
        }  
    }  
}  
  
public class NoCommand implements Command {  
    public void execute() {}  
}
```

Null Object

Command Pattern

```
public void setCommand(int slot, Command oncommand, Command offcommand){
    oncommand[slot] = oncommand;
    offcommand[slot] = offcommand;
}

public void onButtonWasPushed(int slot) {
    oncommand[slot].execute();
}

public void offButtonWasPushed(int slot){
    offcommand[slot].execute();
}
```

Client Program

```
public class RemoteLoader {  
    public static void main(String[] args) {  
        // create invoker  
        RemoteControl remotecontrol = new RemoteControl();  
  
        // create receivers  
        Light livingRoomLight = new Light("Living Room");  
        Light kitchenLight = new Light("Kitchen");  
  
        // create commands  
        Command livingRoomLightOn = new LightOnCommand(livingRoomLight);  
        Command livingRoomLightOff = new LightOffCommand(livingRoomLight);  
        Command kitchenLightOn = new LightOnCommand(kitchenLight);  
        Command kitchenLightOff = new LightOffCommand(kitchenLight);  
  
        // linking the invoker with the commands  
        remotecontrol.setCommand(0, livingRoomLightOn, livingRoomLightOff);  
        remotecontrol.setCommand(1, kitchenLightOn, kitchenLightOff);  
  
        remotecontrol.onButtonWasPushed(0);  
        remotecontrol.offButtonWasPushed(0);  
        remotecontrol.onButtonWasPushed(1);  
        remotecontrol.offButtonWasPushed(1);  
    }  
}
```

After Setting the Commands

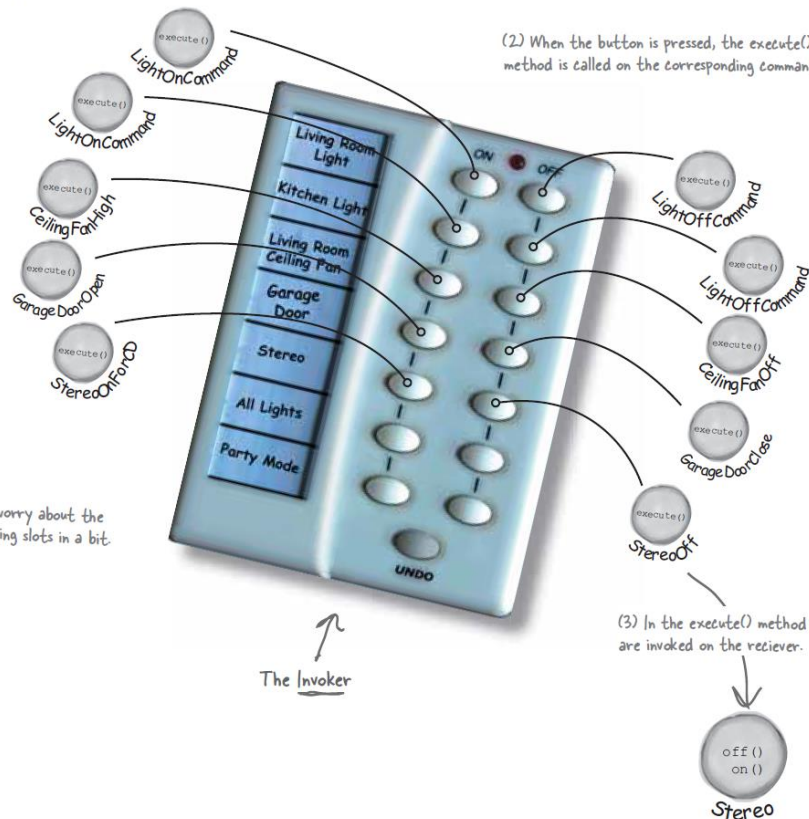
(1) Each slot gets a command.

(2) When the button is pressed, the `execute()` method is called on the corresponding command.

We'll worry about the remaining slots in a bit.

The Invoker

(3) In the `execute()` method actions are invoked on the receiver.



Supporting Undo

```
public class LightOnCommand implements Command {
    Light light;
    public LightOnCommand(Light light){
        this.light = light;
    }
    public void execute() {
        light.on();
    }
    public void undo() {
        light.off();
    }
}

public void onButtonWaspressed(int slot) {
    oncommand[slot].execute();
    undocommand = oncommand[slot];
}

public void offButtonWaspressed(int slot) {
    offcommand[slot].execute();
    undocommand = offcommand[slot];
}

public void undoButtonWaspressed(){
    undocommand.undo();
}
```

Supporting Macro Commands

```
public class MacroCommand implements Command {  
    Command[] command;  
    public MacroCommand(Command[] command){  
        this.command = command;  
    }  
    public void execute(){  
        for(int i = 0; i < command.length; i++)  
            command[i].execute();  
    }  
}
```

Command + Iterator

```
public class MacroCommand implements Command {
    ArrayList <Command> commands;

    public MacroCommand(Command[] commands) {
        this.commands = new ArrayList();
        for (int i=0; i< commands.length; i++)
            this.commands.add(commands[i]);
    }
    public void execute() {
        for (Command item: this.commands)
            item.execute();
    }
    // NOTE: commands have to be done backwards to ensure proper undo
    public void undo() {
        ListIterator<Command> i=commands.listIterator(commands.size());
        while (i.hasPrevious())
            i.previous().undo();
    }
}
```

The left side of the slide features a light blue background with various geometric elements. At the top left, there is a solid blue circle partially cut off by the edge, and next to it, a circle with diagonal white lines. Below these, a vertical white line contains a small circle and a series of dots. Further down, another grid of dots is visible. At the bottom left, a white curved shape with a pink circle on top and two small dots below it is partially shown. A large blue circle with diagonal white lines is also present in the lower left area.

Active Object

Command Interface and Active Object Engine

```
interface Command {  
    public void execute();  
}  
  
class ActiveObjectEngine {  
    LinkedList itsCommands = new LinkedList();  
  
    public void addCommand(Command c) {  
        itsCommands.add(c);  
    }  
  
    public void run() {  
        while (!itsCommands.isEmpty()) {  
            Command c = (Command) itsCommands.getFirst();  
            itsCommands.removeFirst();  
            c.execute();  
        }  
    }  
}
```

Sleep Command

```
class SleepCommand implements Command {
    private Command wakeupCommand = null;
    private ActiveObjectEngine engine = null;
    private long sleepTime = 0; private long startTime = 0;
    private boolean started = false;
    public SleepCommand(long milliseconds, ActiveObjectEngine e, Command wakeupCommand) {
        sleepTime = milliseconds;
        engine = e;
        this.wakeupCommand = wakeupCommand;
    }
    public void execute() {
        long currentTime = System.currentTimeMillis();
        if (!started) {
            started = true;
            startTime = currentTime;
            engine.addCommand(this);
        }
        else if ((currentTime - startTime) < sleepTime ) {
            engine.addCommand(this);
        } else
            engine.addCommand(wakeupCommand);
    }
}
```

DelayedTyper

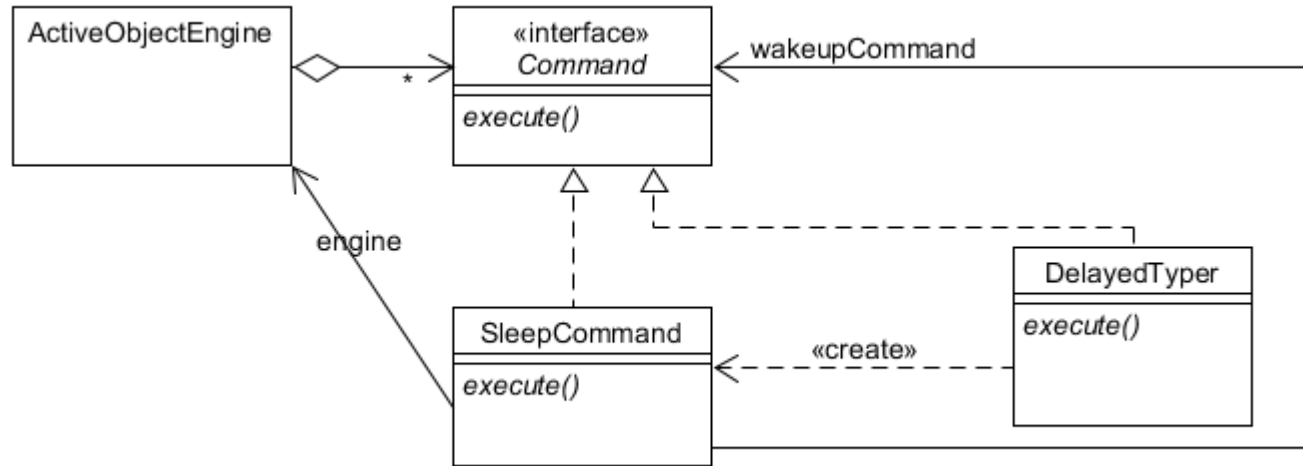
```
public class DelayedTyper implements Command {
    private long mDelay = 0;
    private char mChar;
    private static boolean stop = false;
    private static ActiveObjectEngine engine = new ActiveObjectEngine();
    public DelayedTyper(long delay, char c) {
        mDelay = delay;
        mChar = c;
    }
    @Override public void execute() {
        System.out.print(mChar);
        if (!stop) delayAndRepeat();
    }
    private void delayAndRepeat() {
        engine.addCommand(new SleepCommand(mDelay, engine, this));
    }
    // continued in the next page
}
```

Running the engine

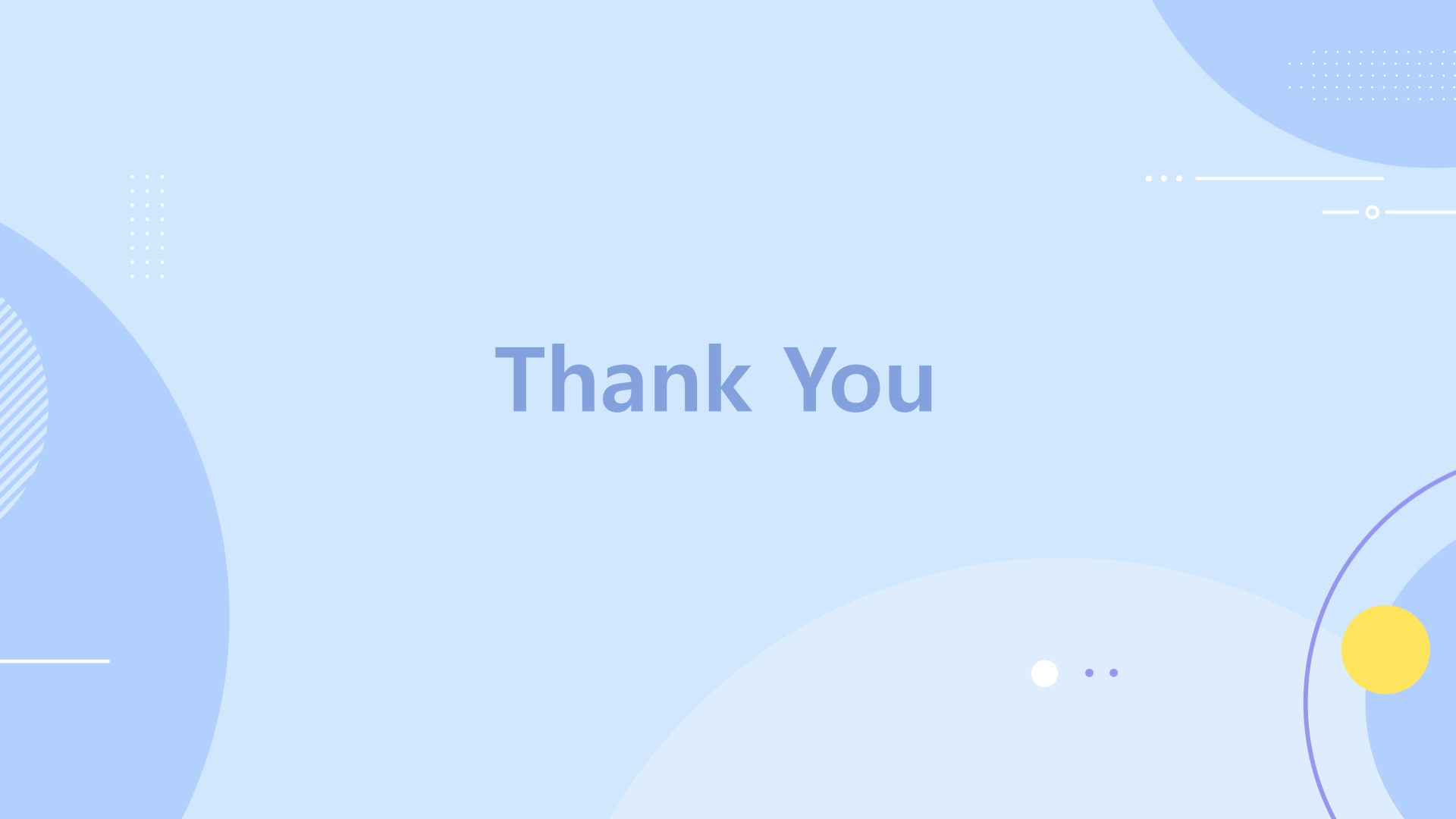
```
public static void main(String args[]) throws Exception {  
    engine.addCommand(new DelayedTyper(100, '1'));  
    engine.addCommand(new DelayedTyper(300, '3'));  
    engine.addCommand(new DelayedTyper(500, '5'));  
    engine.addCommand(new DelayedTyper(700, '7'));  
    Command stopCommand = new Command() {  
        @Override public void execute() { stop = true; }  
    };  
    engine.addCommand(new SleepCommand(10000, engine, stopCommand));  
    engine.run();  
}  
} // end of class
```

```
13571113115131711315113117351111315137111131513117135111131571131113151713111351111371151311  
13157113111351171311513111371511311351171131511311137511311113571113151311173151357
```

Class Diagram for Active Object Engine



- Command **decouples** the **object that invokes the operation** from the **one that knows how to perform it**.
- Commands can be manipulated and extended like any other object.
- You can assemble commands into a composite command.
 - E.g. MacroCommand class
 - In general, composite commands are an instance of Composite pattern

The background is a light blue gradient. It features several abstract geometric elements: a large blue circle on the left with a white horizontal line; a blue circle in the top right with a white dotted pattern; a blue circle in the bottom right with a yellow circle inside it; and a large light blue circle at the bottom. There are also white dotted patterns in the top left and bottom right, and a white horizontal line with a small circle in the top right.

Thank You

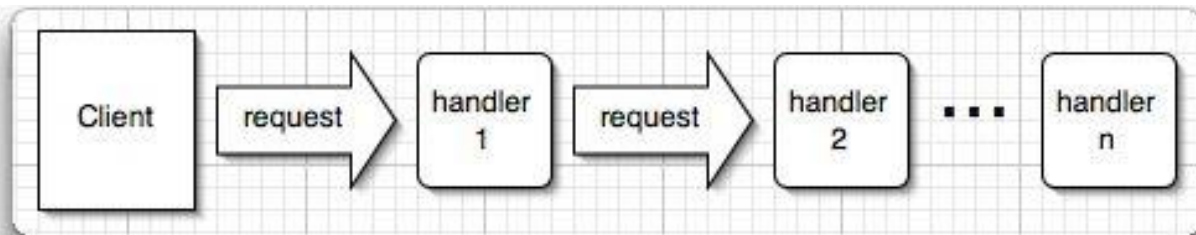
A vertical blue sidebar on the left side of the slide. It contains several decorative elements: a large blue circle at the top left with a white dotted pattern; a smaller blue circle at the top right with a white diagonal line pattern; a vertical white line with a small white circle in the middle and three small white dots at the bottom; a large blue circle at the bottom left with a white dotted pattern; a smaller blue circle at the bottom right with a white diagonal line pattern; and a white curved shape at the bottom left with a small red circle and three small blue dots.

02

Chain-of-Responsibility Pattern

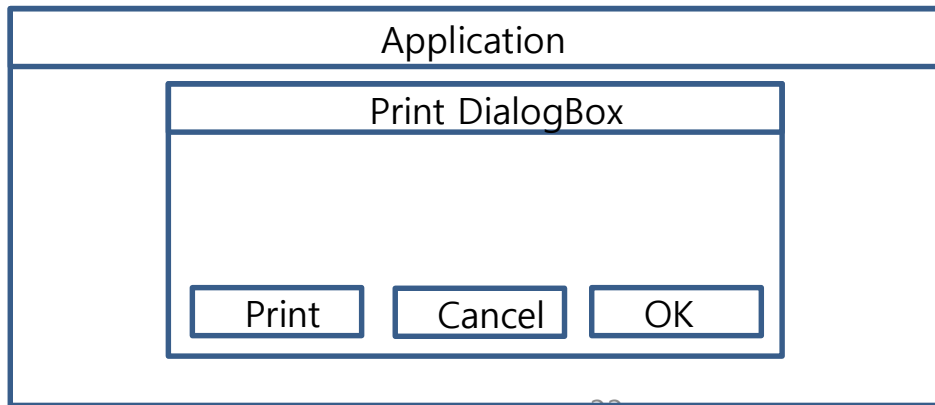
Chain of Responsibility Pattern

- Purpose
 - Gives more than one object an opportunity to handle a request by linking receiving objects together.
- Use When
 - Multiple objects may handle a request and the handler doesn't have to be a specific object.
 - A set of objects should be able to handle a request with the handler determined at runtime.
 - A request not being handled is an acceptable potential outcome.



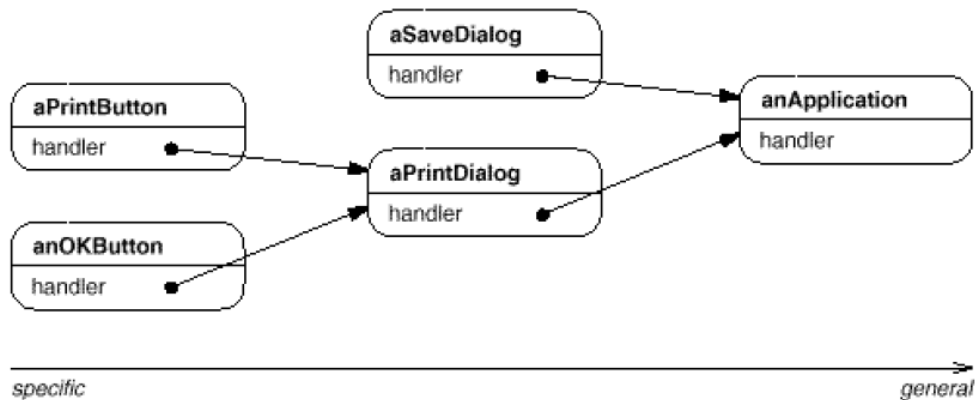
Example

- A context sensitive **help facility** for a GUI
 - The user can obtain help information on any part of the interface just by clicking on it.
 - The help that is provided depend on the parts of the interface that's selected and its context.
 - Sometimes we will encounter with help about a particular button, a particular dialog, or the application in general based on whether help is available on that subject.



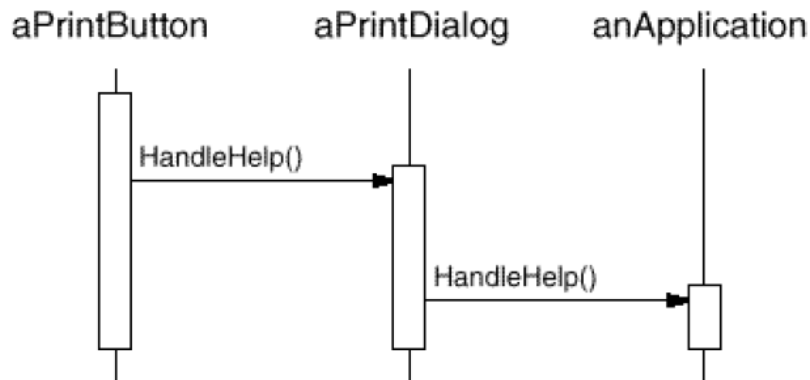
Example (Cont)

- The help information is organized according to its generality - from the most specific to the most general
- The problem is that the object that ultimately provides the help is not known explicitly to the object that initiates the help request
- We need a way to decouple the button that initiates the help request from the objects that provide help information



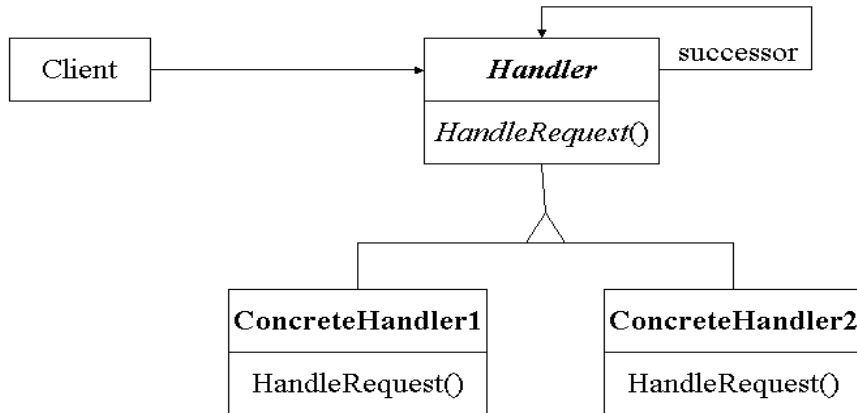
Chain of Responsibility Pattern

- Each object in the chain
 - receives the request
 - handle it or forward it to the next object
- Notice
 - Object making the request has no knowledge of which object is handling the request
 - The request has an implicit receiver

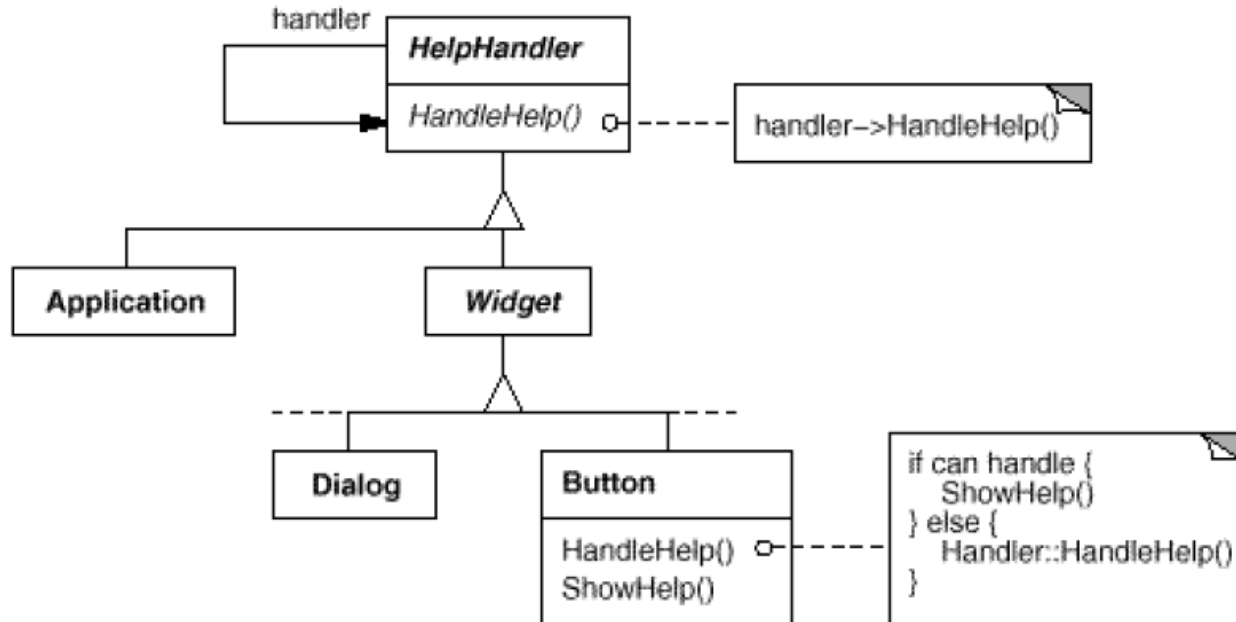


HelpHandler class diagram

- Each object in the chain **shares a common interface for handling requests and accessing its successor** on the chain.



COR in Practice



Participants

- Handler
 - defines an interface for handling the requests
 - (optional) implements the successor link
- Concrete Handler
 - handles requests it is responsible for
 - can access its successor
 - if the Concrete Handler can handle the request, it does so; otherwise it forwards the request to its successor
- Client
 - initiates the request to a Concrete Handler object on the chain

Consequences

- Benefits:
 - Decoupling of senders and receivers
 - Added flexibility
 - Sender doesn't need to know specifically who the handlers are
- Potential Drawbacks:
 - Client can't explicitly specify who handles a request
 - No guarantee of request being handled (request falls off end of chain)

Implementation

- Implementing the successor chain
 - Option 1: Define new links (in Handler or ConcreteHandler)
 - Option 2: Use existing links
(such as the parent references from the Composite pattern)
- Connecting successors
- Representing requests

```
abstract class Handler {  
    protected Handler successor;  
  
    public void setSuccessor(Handler successor) {  
        this.successor = successor;  
    }  
  
    public abstract void HandleRequest(int request);  
}
```

Implementation(handler 1)

```
class ConcreteHandler1 extends Handler {  
    public void HandleRequest(int request) {  
  
        if (request >= 0 && request < 10)  
            System.out.println("ConcreteHandler1 handled request " + request);  
        else if (successor != null)  
            successor.HandleRequest(request);  
  
    }  
}
```

Implementation(handler 2)

```
class ConcreteHandler2 extends Handler {  
    public void HandleRequest(int request) {  
  
        if (request >= 10 && request < 20)  
            System.out.println("ConcreteHandler2 handled request " + request);  
        else if (successor != null)  
            successor.HandleRequest(request);  
  
    }  
}
```

Implementation(handler 3)

```
class ConcreteHandler3 extends Handler {  
    public void HandleRequest(int request) {  
  
        if (request >= 20 && request < 30)  
            System.out.println("ConcreteHandler3 handled request " + request);  
        else if (successor != null)  
            successor.HandleRequest(request);  
  
    }  
}
```

Implementation (main app)

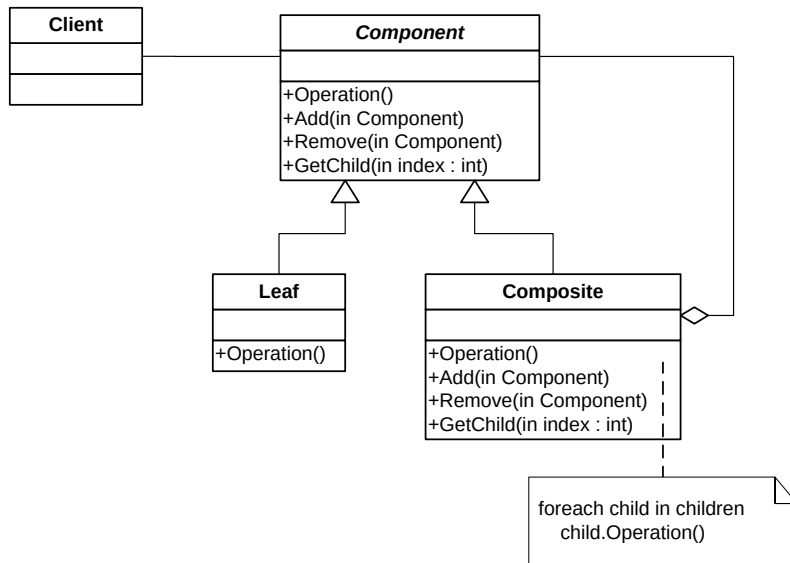
```
class MainApp {  
  
    /// <summary>  
    /// Entry point into console application.  
    /// </summary>  
  
    public static void main() {  
  
        // Setup Chain of Responsibility  
        Handler h1 = new Concrete handler1();  
        Handler h2 = new Concrete handler2();  
        Handler h3 = new Concrete handler3();  
        h1.SetSuccessor(h2);  
        h2.SetSuccessor(h3);  
  
        // Generate and process request  
        int[] requests = new []  
            {2, 5, 14, 22, 18, 3, 27, 20};  
  
        for (int request in requests) {  
            h1.handleRequest(request);  
        }  
  
    }  
}
```

Output

```
ConcreteHandler1 handled request 2  
ConcreteHandler1 handled request 5  
ConcreteHandler2 handled request 14  
ConcreteHandler3 handled request 22  
ConcreteHandler2 handled request 18  
ConcreteHandler1 handled request 3  
ConcreteHandler3 handled request 27  
ConcreteHandler3 handled request 20
```

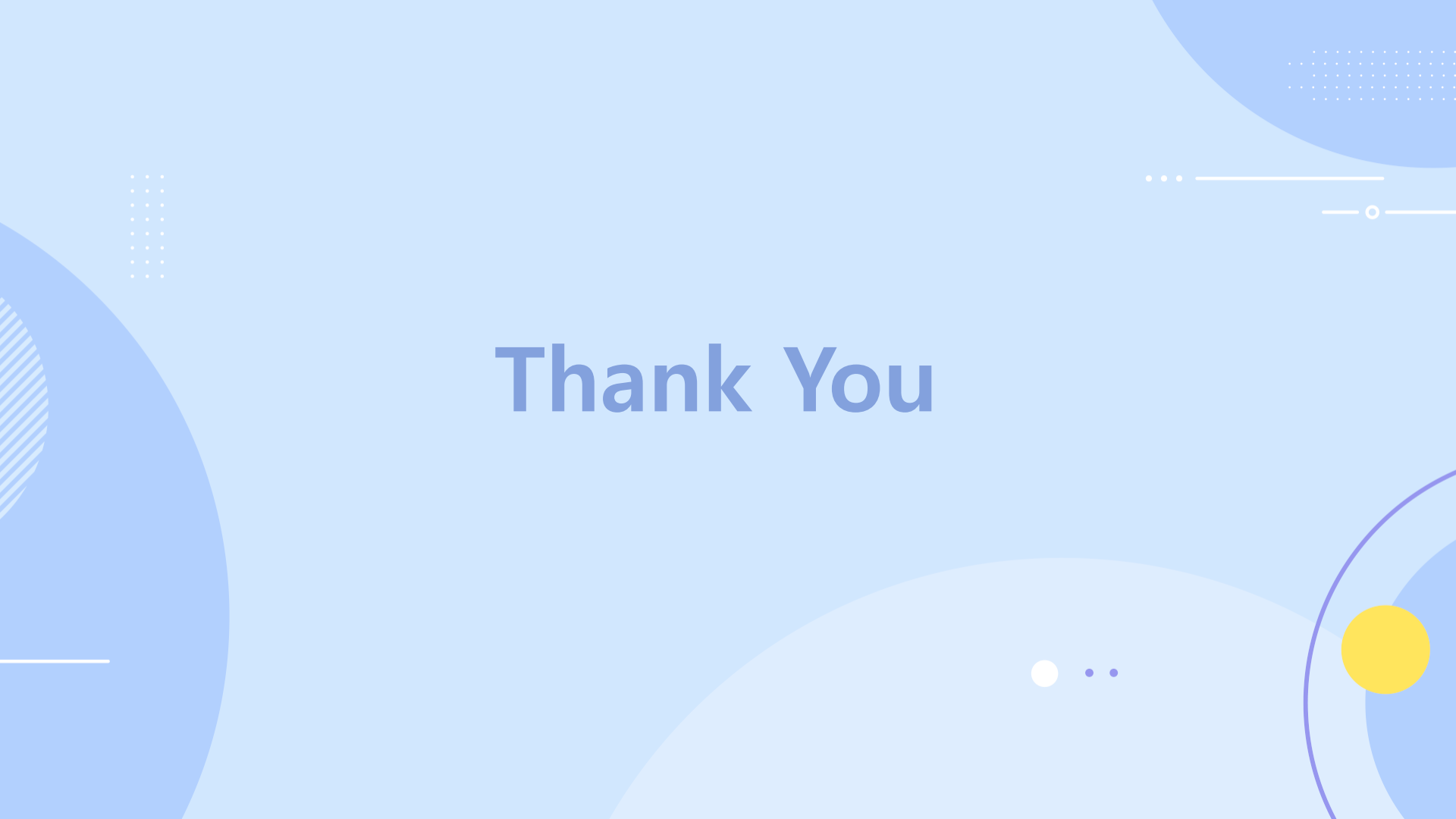
Review - Related pattern

- Chain of responsibility is often applied in conjunction with Composite.
 - Think of the "Help" example



Review - Related pattern

- Chain of Responsibility, Command, Mediator, and Observer, address how you can decouple senders and receivers, but with different tradeoffs.
 - Chain of Responsibility passes a sender request along a chain of potential receivers.
- Chain of Responsibility can use Command to represent requests as objects.



Thank You

A vertical blue sidebar on the left side of the slide. It features several overlapping circles and semi-circles. Some are solid blue, while others have patterns of white dots or diagonal white lines. A white vertical line with a small circle in the middle and dots at the bottom is also present. At the bottom left, there is a white semi-circle with a small red circle on its edge and two small dark blue dots below it.

03

Prototype Pattern

Prototype Pattern

- **Purpose**

- Create objects based upon a template of an existing objects through cloning.

- **Use When**

- Composition, creation, and representation of objects should be decoupled from a system.
- Classes to be created are specified at runtime.
- Objects or object structures are required that are identical or closely resemble other existing objects or object structures.
- The initial creation of each object is an expensive operation.
- When to avoid building a class hierarchy of factories that parallels the class hierarchy of products

Class Diagram for Abstract Factory Pattern

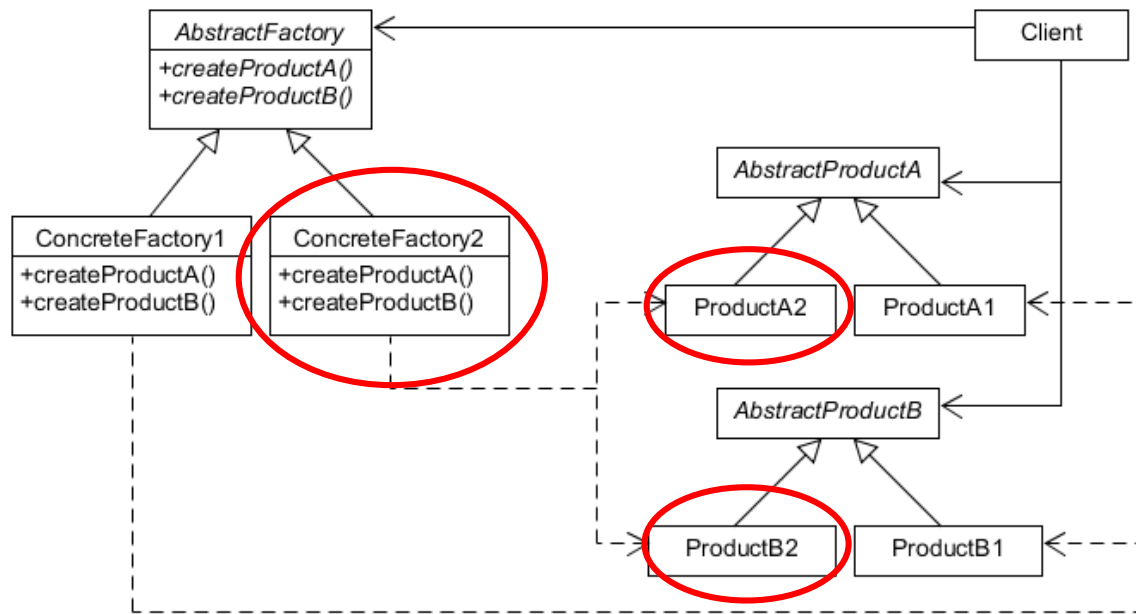
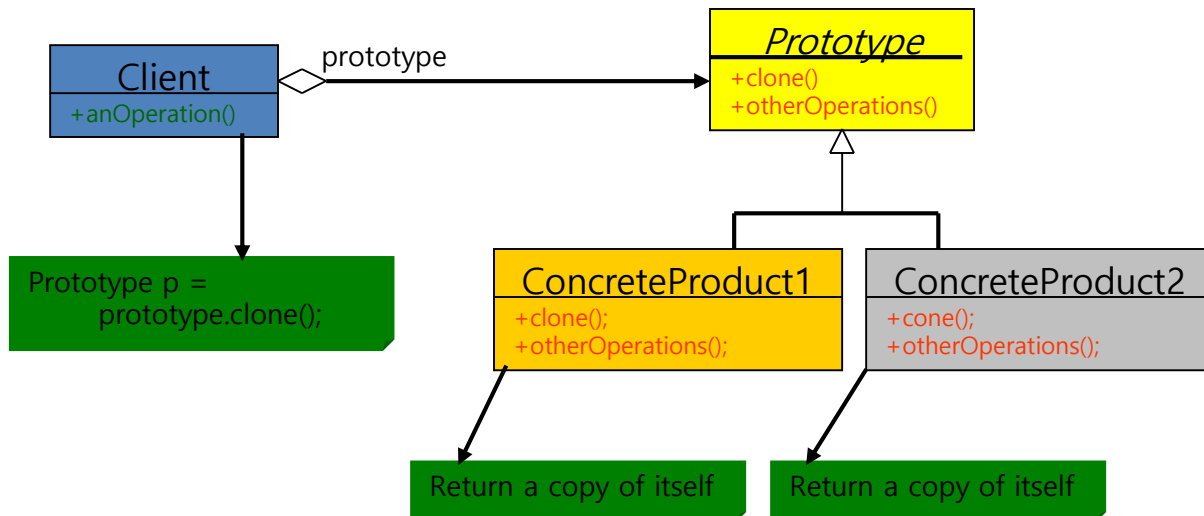
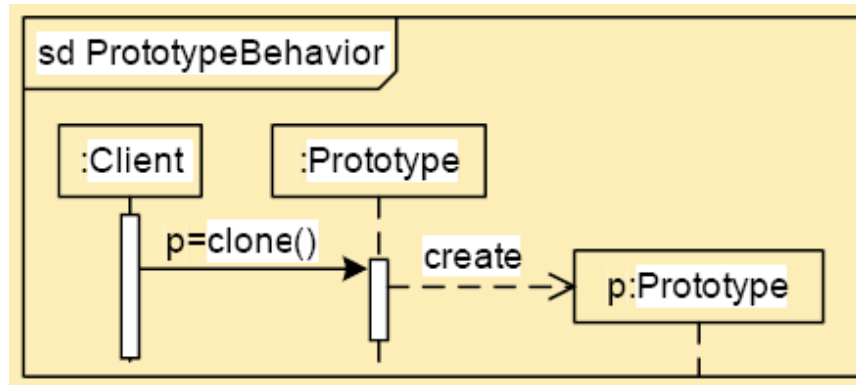


Figure from [GOF]

Prototype Pattern in Class Diagram



Prototype Pattern in Sequence Diagram



The client asks the prototype to clone itself for a new object of the prototype.

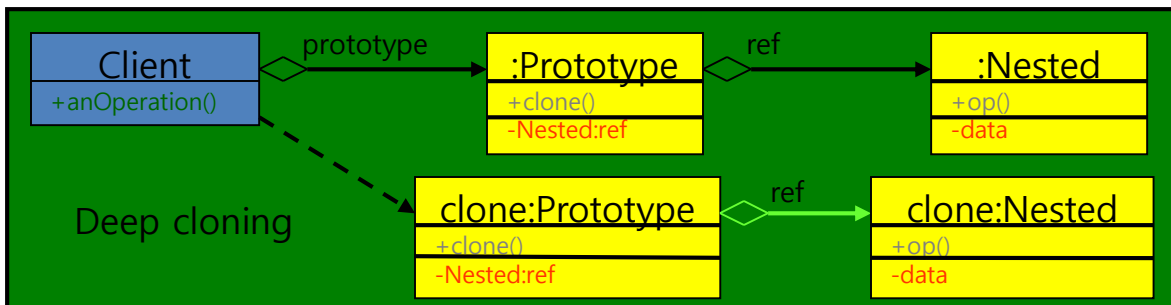
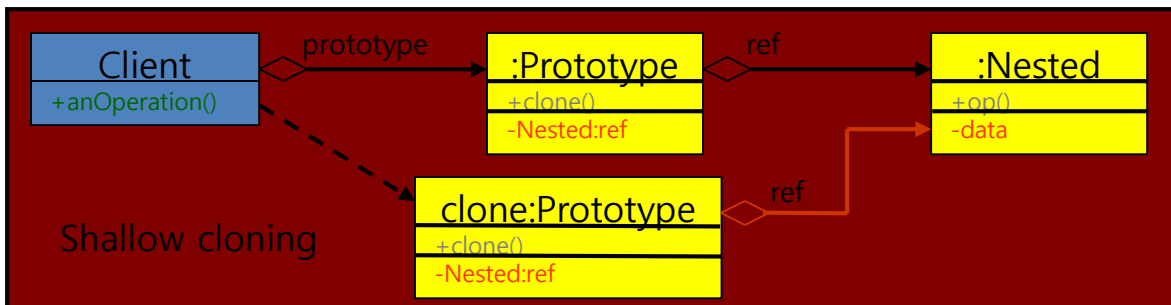
Participants

- Prototype
 - defines the interface (an operation) of cloning itself.
- ConcreteProduct1 and ConcreteProduct2
 - Concrete objects that can clone themselves.
- Client
 - Obtain more objects by asking them to clone themselves.

What's so special about the pattern?

- Advantages of creational patterns
 - Hides concrete product classes from clients
 - Decouples the clients from the creational process
- Unique feature of Prototype
 - Prototypes can be **supplied and changed at runtime**
 - Thus, it provides **great flexibility** in configuring and changing a program at runtime
 - Adding and removing products at run-time
 - Reduced subclassing
 - Configuring an application with classes dynamically. Loading the classes dynamically

Shallow vs. deep cloning (coping)



Cloning in Java

- Java provides support for cloning with Cloneable interface and a protected method clone method in the Object class.
- Any class using the built-in cloning mechanism is supposed to
 - Implement the Cloneable interface
 - Define a concrete public or protected clone() operation
 - In the clone() operation, obtain a new object by calling super.clone()
- Default clone() operation makes a shallow copy!

Using Java's Cloneable interface (Shallow)

```
//Shallow cloning: Java's Cloneable
class Stack implements Cloneable {
    private final int maxSize;
    private int top;
    private Object[] store;

    public Stack (int size) {
        store = new Object[maxSize = size];
        top = -1;
    }

    public Object clone() throws CloneNotSupportedException {
        try { return super.clone(); }
        catch (CloneNotSupportedException e) { return null; }
    }
    // ....
}
```

Using Java's Cloneable interface (Deep)

```
//Deep cloning: Java's Cloneable
class Stack implements Cloneable {
    private final int maxSize;
    private int top;
    private Object[] store;

    public Stack (int size) {
        store = new Object[maxSize = size];
        top = -1;
    }

    public Object clone() throws CloneNotSupportedException {
        Stack result;
        try {
            result = (Stack) super.clone();
            result.store = (Object[])store.clone();
            return result;
        }
        catch (CloneNotSupportedException e) { return null; }
    }
    // ....
}
```

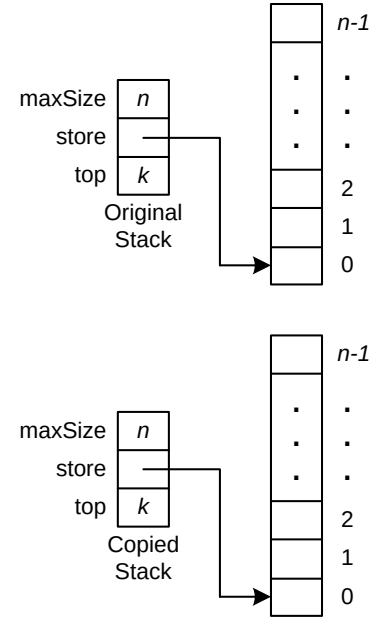
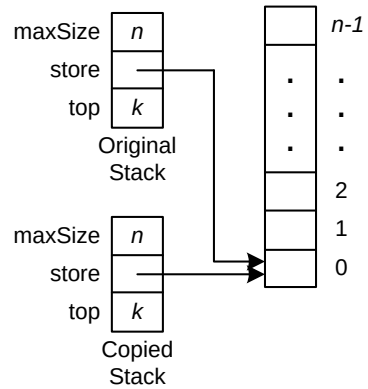
Not Using Java's Cloneable interface (Deep)

```
//Deep cloning: Not using Java's Cloneable
class Stack {
    private final int maxSize;
    private int top;
    private Object[] store;

    public Stack (int size) {
        store = new Object[maxSize = size];
        top = -1;
    }

    public Stack CreateClone() {
        Stack result = new Stack(maxSize);
        result.top = top;
        for (int k = 0; k < size; k++)
            result.store[k] = store[k];
        return result;
    }
    // ....
}
```

Shallow vs. Deep cloning

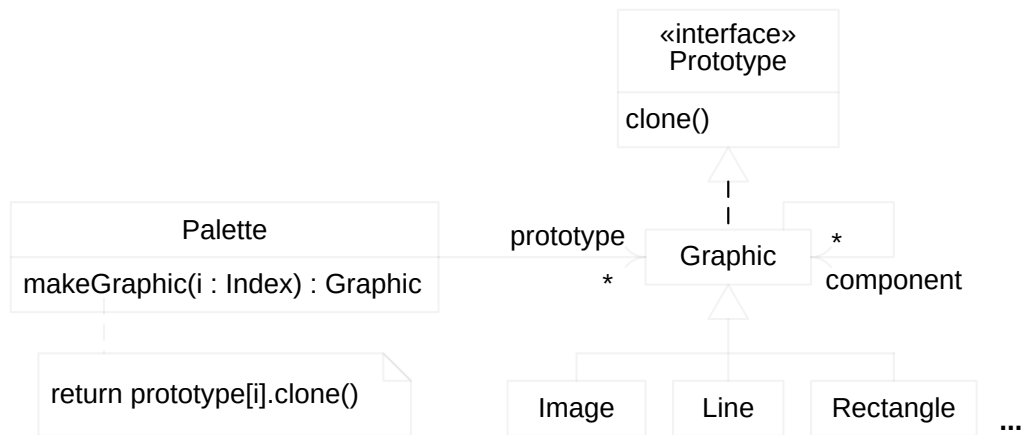


Example

- A software architecture editor
 - Handle box-and-line diagrams and UML notations
 - Allow users to invent new icons or connectors and add them to a palette for later use
 - New graphic entities can be created by users and have no defining class
 - graphic **primitives** (rectangles, lines, and images) or **composites** made up of other graphic elements
 - Realization through Prototype
 - Let the palette hold prototypes of the new graphic objects and clone them when a user wants a new one

Example

- All graphic objects in the editor must be cloneable
- Allow to select any graphic object as prototype.



Example explained

- **Graphic** object may be
 - a **primitive entity** (e.g. a **Line**) or a **composite entity with components**
- Graphic objects have a **done()** operation:
 - Step 1: User requests new object from the **Palette** => calls **makeGraphic()** operation
 - Step 2: **makeGraphic()** asks the requested object to done itself
- New items can be added to the **Palette** or old ones can be removed at any time
- Note: **Palette** does not need to know the classes of the prototypes it manages because it does not use a constructor to make them
 - **Palette** (a client) is thus decoupled from the products it uses by the **Prototype** instances it manages (the generators)

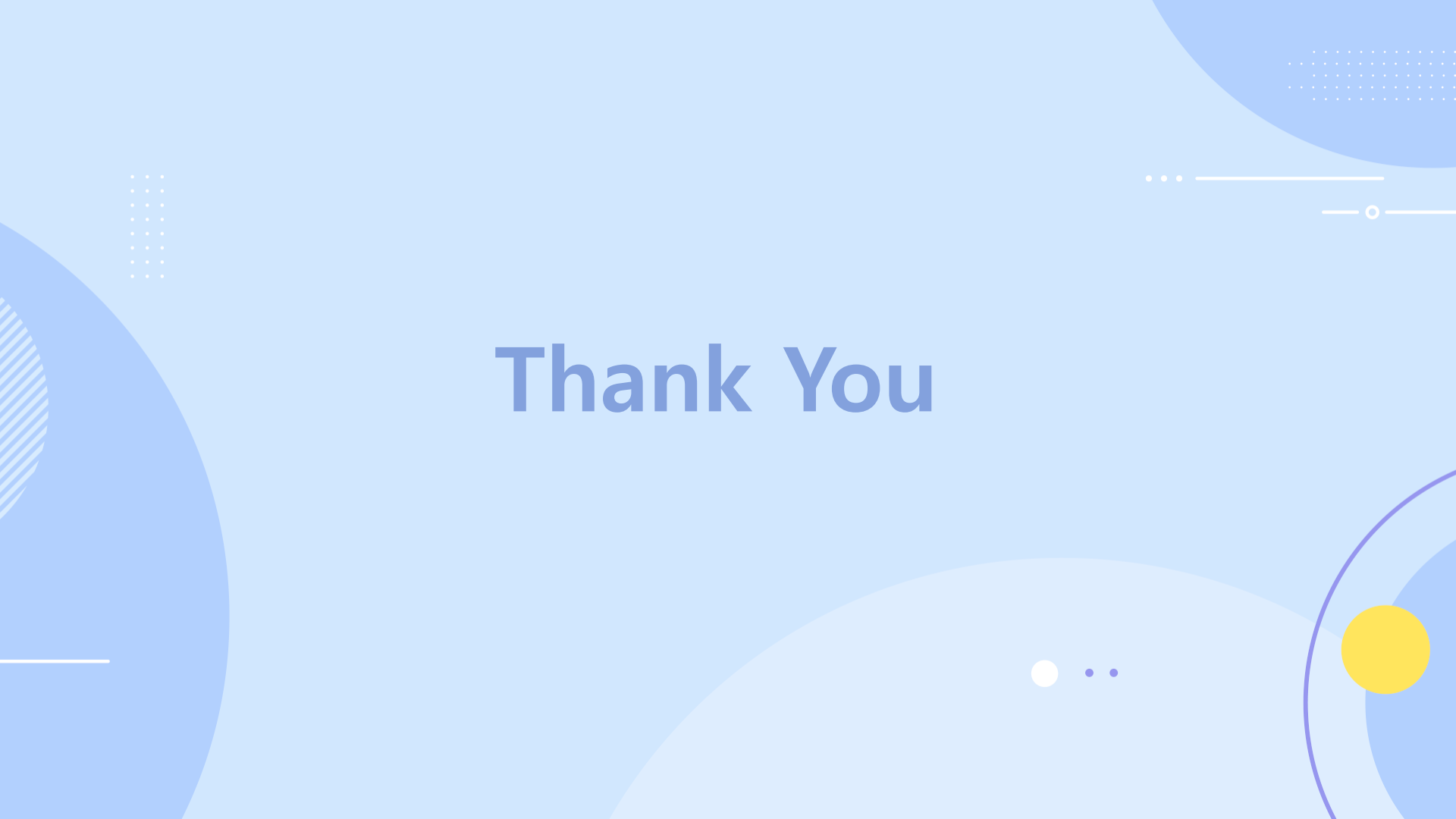
Working with Abstract Factory Pattern

- Abstract Factory classes are often implemented with Factory Methods, but they can be implemented using Prototype.
 - Factory Method: creation through inheritance.
 - Prototype: creation through delegation.

```
class MazeFactory {  
    Maze prototypeMaze;  
    Wall prototypeWall;  
    Room prototypeRoom;  
    Door prototypeDoor;  
    MazeFactory (Maze m, Wall w, Room r, Door d) {  
        prototypeMaze = m; prototypeWall = w;  
        prototypeRoom = r; prototypeDoor = d;  
    }  
    Wall makewall () { return prototypeWall.clone(); }  
    void setWallPrototype (Wall w) { prototypeWall = w; }  
    Door makeDoor (Room r1, Room r2) {  
        Door door = prototypeDoor.clone();  
        door.initialize(r1, r2);  
        return door;  
    }  
}
```

Review

- Be aware of the shallow cloning VS deep cloning
- Prototypes are useful when object initialization is expensive, and you anticipate few variations on the initialization parameters. In this context, Prototype can avoid expensive "creation from scratch", and support cheap cloning of a pre-initialized prototype.
- Abstract Factory and Prototype can be used together
 - Abstract Factory can store set of prototypes which are cloned to return product objects
- Very useful with Composite and Decorator patterns



Thank You

A vertical blue sidebar on the left side of the slide. It contains several decorative elements: a circle with diagonal lines at the top, a vertical line with a small circle in the middle and three dots at the bottom, a grid of small dots, and a large circle with diagonal lines at the bottom. A small pink circle is also visible near the bottom left.

04

Proxy Pattern

- **Purpose**

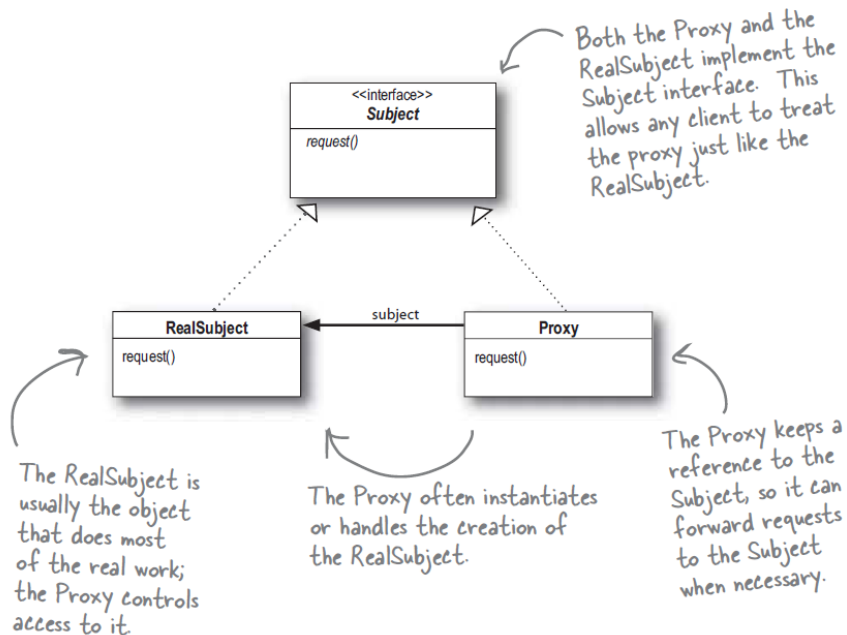
- Allows for object level access control by acting as a pass through entity or a placeholder object.

- **Use When**

- Access control for the original object is required.
- Added functionality is required when an object is accessed.

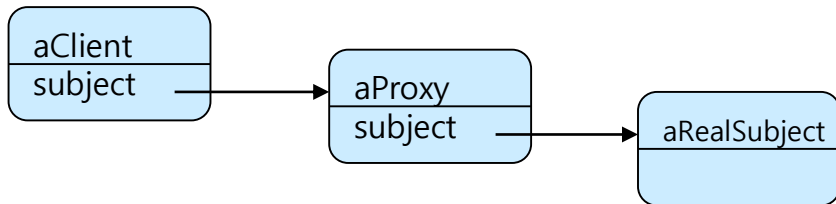
The Proxy Pattern

- Proxy Pattern provides a surrogate or placeholder for another object to control access to it



Proxy Pattern Collaborations

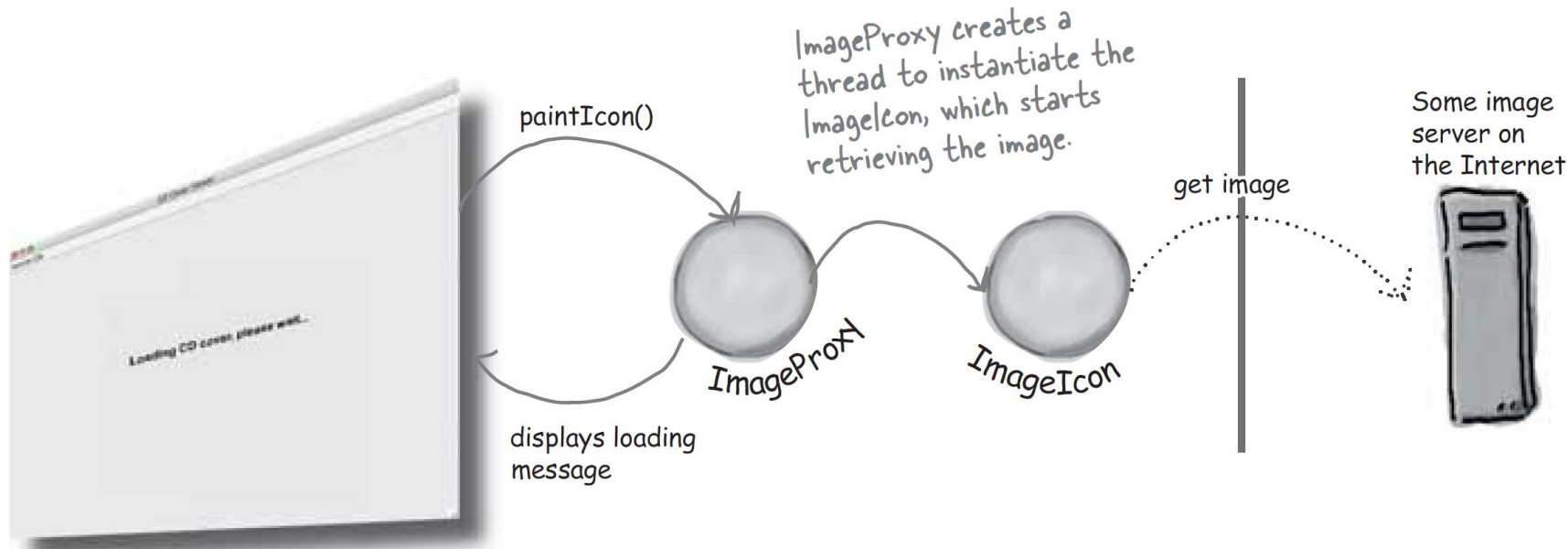
- Subject
 - defines the common interface for RealSubject and Proxy so that a Proxy can be used anywhere a RealSubject is expected
- RealSubject
 - defines the real object that the proxy represents
- Proxy
 - maintains a reference that lets the proxy access the real subject
 - provides an interface identical to Subject's so that a proxy can be substituted for the real subject
 - controls access to the real subject and may be responsible for creating and deleting it



- Need for a more versatile or sophisticated reference to an object than a simple pointer
 - Remote proxy
 - Responsible for encoding a request and its arguments and for sending the encoded request to the **real subject in a different address space**
 - Virtual proxy
 - may **cache additional information about the real subject** so that they can postpone accessing it
 - Protection proxy
 - **checks that the caller has the access permissions** required to perform a request

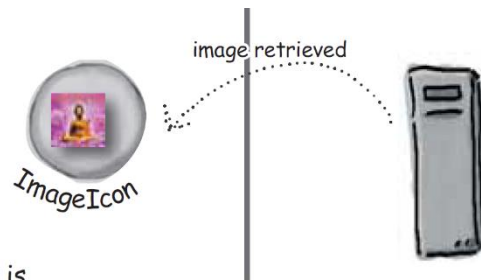
Virtual Proxy in Action

- 1 We created an ImageProxy for the display. The `paintIcon()` method is called and ImageProxy fires off a thread to retrieve the image and create the ImageIcon.

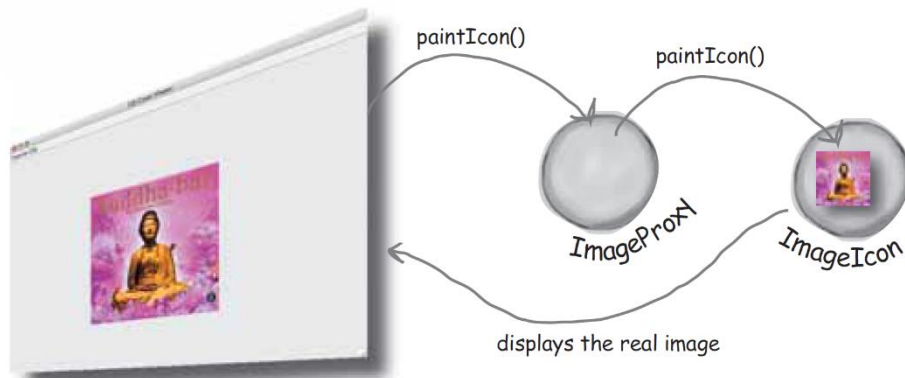


Virtual Proxy in Action

- 2 At some point the image is returned and the ImageIcon fully instantiated.

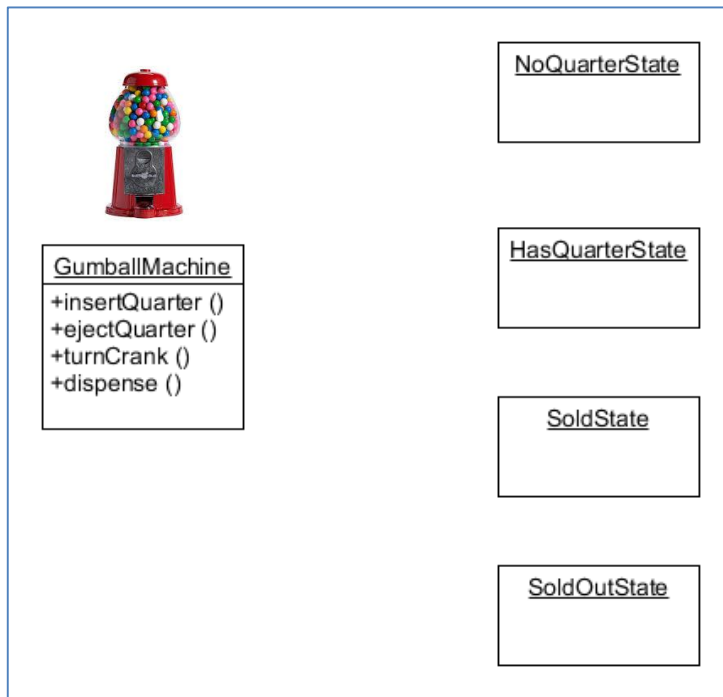


- 3 After the ImageIcon is created, the next time paintIcon() is called, the proxy delegates to the ImageIcon.



Request From the CEO of Mighty Gumball

- He wants to have some monitoring program for his gumball machines.
 - getCount()
 - getState()
 - getLocation()



Coding the Monitor

```
public class GumballMachine {  
    // other instance variables  
    String location;  
    public GumballMachine(String location, int count) {  
        // other constructor code here  
        this.location = location;  
    }  
    //other methods here  
}
```

```
public class GumballMonitor {  
    GumballMachine machine;  
    public GumballMonitor(GumballMachine machine) {  
        this.machine = machine;  
    }  
    public void report() {  
        System.out.println("Gumball Machine: " + machine.getLocation());  
        System.out.println("Current inventory: " + machine.getCount() + " gumballs");  
        System.out.println("Current state: " + machine.getState());  
    }  
}
```

Testing the Monitor

```
public class GumballMachineTestDrive {
    public static void main(String[] args) {
        int count = 0;
        if (args.length < 2) {
            System.out.println("GumballMachine <name> <inventory>");
            System.exit(1);
        }
        try {
            count = Integer.parseInt(args[1]);
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
        GumballMachine gumballMachine = new GumballMachine(args[0], count);
        GumballMonitor monitor = new GumballMonitor(gumballMachine);
        // rest of test code here

        monitor.report();
    }
}
```

Was the CEO Satisfied?

File Edit Window Help FlyingFish

```
% java GumballMachineTestDrive Seattle 112
```

```
Gumball Machine: Seattle
```

```
Current Inventory: 112 gumballs
```

```
Current State: waiting for quarter
```

Well, what the CEO really wanted was to monitor the gumball machines REMOTELY!

Distributed Computing

- Distributed Computing
 - involves the design and implementation of applications as a set of cooperating software entities (processes, threads, objects) that are distributed across a network of machines
- Advantages to Distributed Computing
 - Performance
 - Scalability
 - Resource Sharing
 - Fault Tolerance
- Difficulties in developing Distributed Computing systems
 - Latency
 - Synchronization
 - Partial Failure

Client-Server Model and Programming

- Client-Server Model
 - Client - entity that makes a request for a service
 - Server - entity that responds to a request and provides a service
 - The predominant networking protocol in use today is the Internet Protocol (IP). The main API for writing client-server programs using IP is the Berkeley socket API.
- Programming
 - Dealing with all of the details of the socket library calls can be tedious. (See, for example, Stevens' *Unix Network Programming*.)
 - The java.net package provides classes to abstract away many of the details of socket-level programming, making it simple to write client-server applications

Remote Procedure Call (RPC)

- Disadvantage of Client-Server model
 - Both the client and server had to be aware of the socket level details
- Wouldn't it be nice if even these details were abstracted away and the request to the server looked like a local procedure call from the viewpoint of the client?
 - That's the idea behind a Remote Procedure Call (RPC), a technology introduced in the late 1970's
- Two RPC specifications:
 - SUN's Open Network Computing (ONC) RPC
 - OSF's Distributed Computing Environment (DCE) RPC

Distributed Object Technology

- But RPC is not object-oriented. In the OO world, we'd like to have distributed objects and remote method calls.
- Remote Method Invocation (RMI)
 - Developed by SUN
 - Available as part of the core Java API
 - Java-centric
 - Object interfaces defined as Java interfaces
 - Uses object serialization

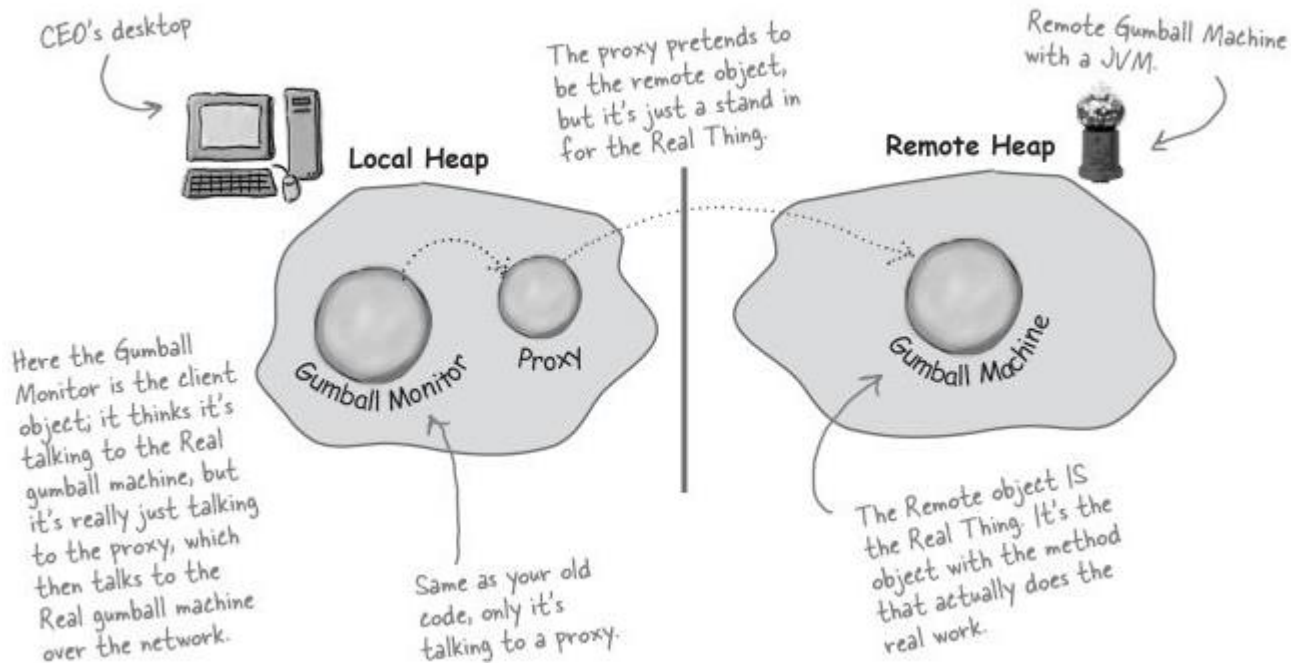
Remote Method Invocation

- Provides a distributed object capability for Java applications
- Allows a Java method to obtain a reference to a remote object and invoke methods of the remote object nearly as easily as if the remote object existed locally
- The remote object can be in another JVM on the same host or on different hosts across the network
- Uses object serialization to marshal and unmarshal method arguments
- Supports the dynamic downloading of required class files across the network

RMI Stubs and Skeletons

- RMI uses stub and skeleton objects to provide the connection between the client and the remote object
- *A stub is a proxy for a remote object which is responsible for forwarding method invocations from the client to the server where the actual remote object implementation resides*
- A client's reference to a remote object, therefore, is actually a reference to a local stub. The client has a local copy of the stub object.
- *A skeleton is a server-side object which contains a method that dispatches calls to the actual remote object implementation*
- A remote object has an associated local skeleton object to dispatch remote calls to it

Using Remote Proxy

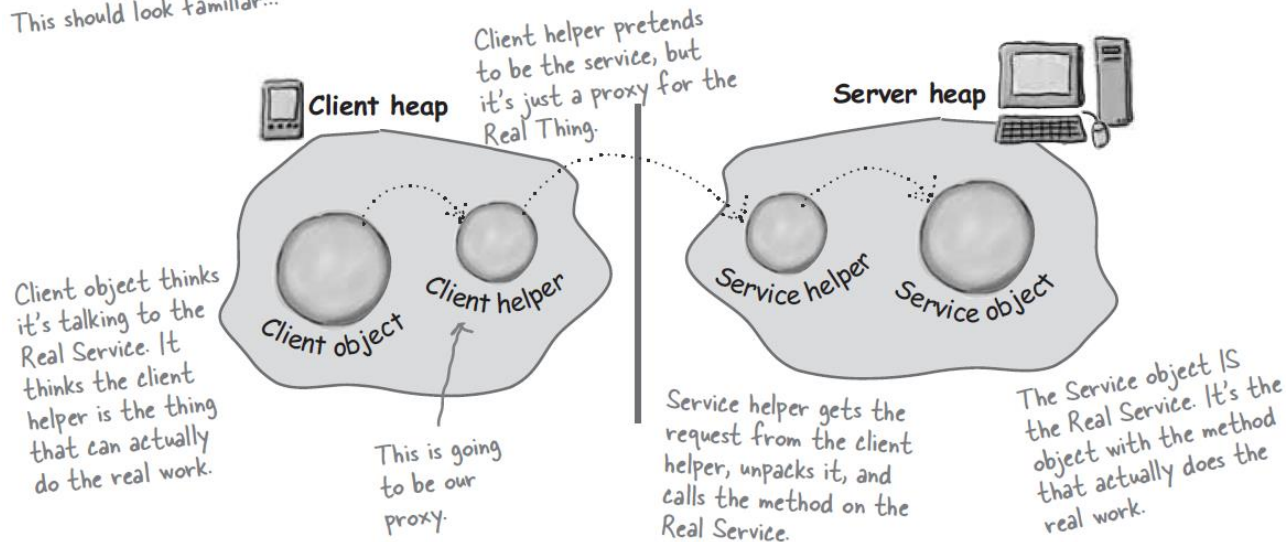


The Roles of Helper Objects

- Request From Client to Server
 - Client Side: The client calls a method on the client helper, as if the client helper were the actual service. Then the client helper takes care of forwarding that request to the server.
 - Server Side: The service helper receives the request from the client helper, via a Socket connection, unpacks the information about the call and then invokes the real method on the real service object. (local call in the server)
- Reply From Server to Client
 - Server Side: The service helper gets the return value from the service, packs it, and ships it back to the client helper.
 - Client Side: The client helper unpacks the information and returns the value to the client object.

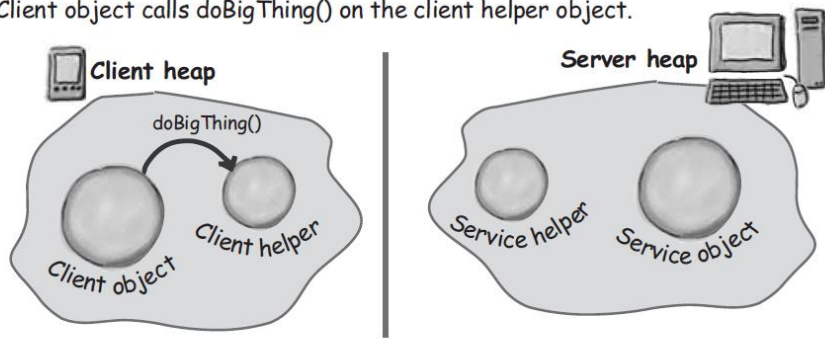
Remote Method Invocation

This should look familiar...

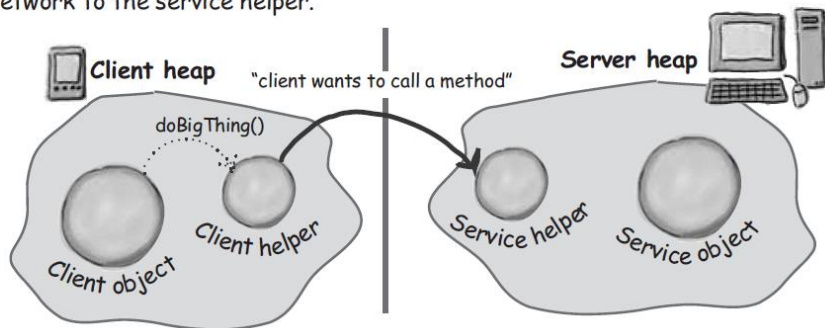


How the Method Call Happens

- ① Client object calls `doBigThing()` on the client helper object.

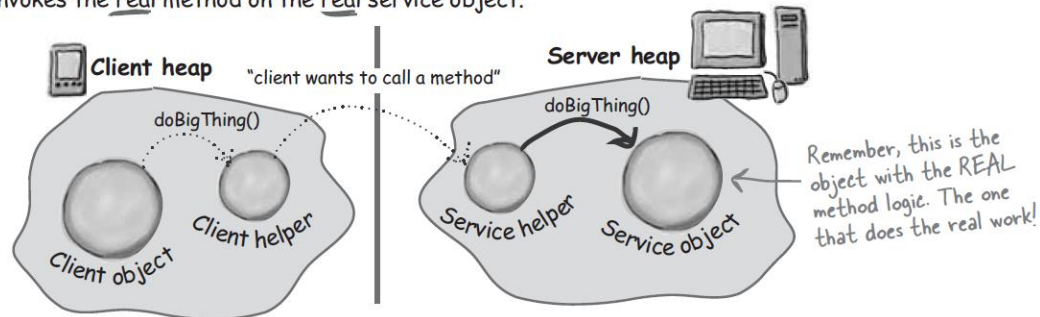


- ② Client helper packages up information about the call (arguments, method name, etc.) and ships it over the network to the service helper.

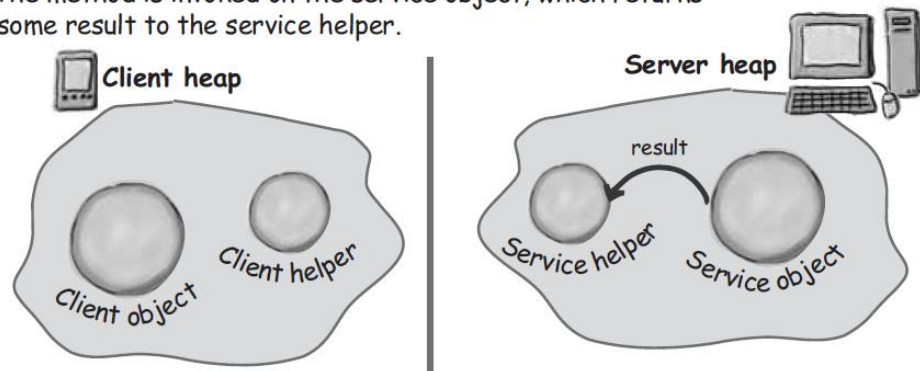


How the Method Call Happens

- ③ Service helper unpacks the information from the client helper, finds out which method to call (and on which object) and invokes the real method on the real service object.

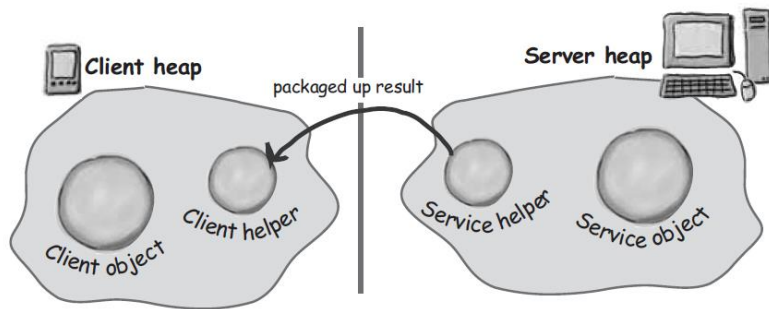


- ④ The method is invoked on the service object, which returns some result to the service helper.

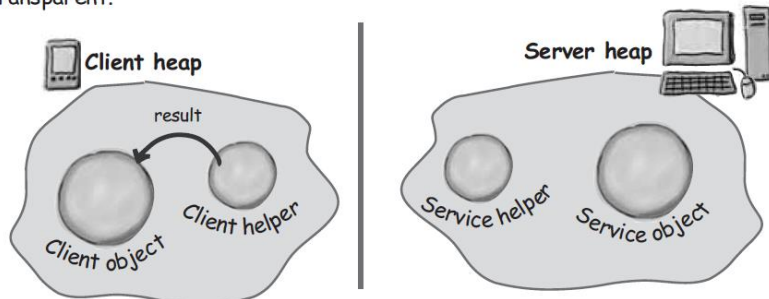


How the Method Call Happens

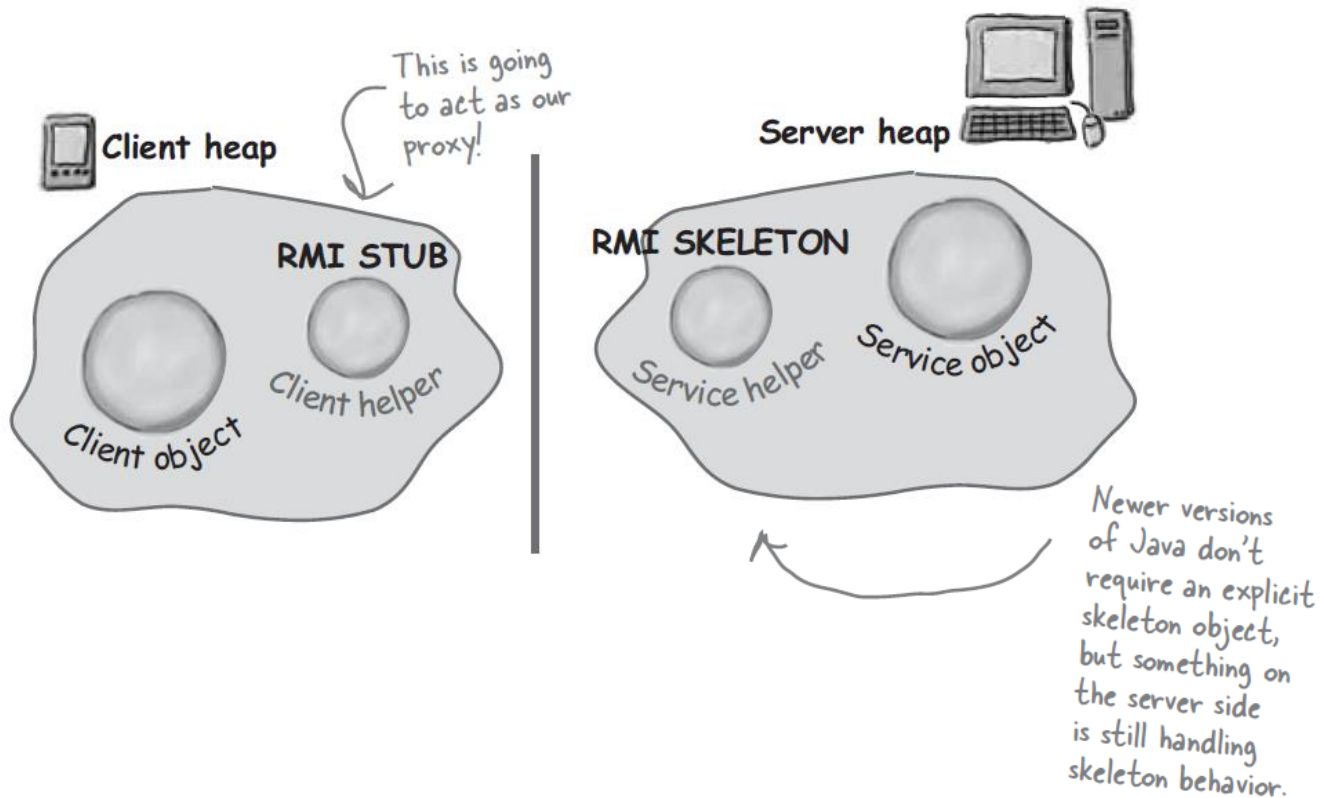
- ⑤ Service helper packages up information returned from the call and ships it back over the network to the client helper.



- ⑥ Client helper unpackages the returned values and returns them to the client object. To the client object, this was all transparent.



Java RMI the Big Picture



Making the Remote Service

1. Make a Remote Interface (MyService.java)

- The remote interface defines the methods that a client can call remotely

2. Make a Remote Implementation (MyServiceImpl.java)

- It has the real implementation of the remote methods defined in the remote interface

3. Generate the stubs and skeletons using rmic

- Helpers <- they are automatically generated from the remote implementation, MyServiceImpl.java.

4. Start the RMI Registry

5. Start the remote service

Step One: Make a Remote Interface

- Extend `java.rmi.Remote`
- Declare the all methods throw a `RemoteException`
- Be sure arguments and return values are primitives or `Serializable`

```
import java.rmi.*;  
public interface MyRemote extends Remote {  
    public String sayHello() throws RemoteException;  
}
```

Step Two: Make a Remote Implementation

- Implement the Remote Interface
- Extend `UnicastRemoteObject`
- Write a no-arg constructor that declares a `RemoteException`
- Register the service with the RMI registry

Remote Service Implementation

```
import java.rmi.*;
import java.rmi.server.*;
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
    public String sayHello() { return "Server says, Hey"; }
    public MyRemoteImpl() throws RemoteException { }
    public static void main (String[] args) {
        try {
            MyRemote service = new MyRemoteImpl();
            Naming.rebind("RemoteHello", service);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

Step Three: Generates Stubs and Skeletons

- Run *rmic* on the remote implementation class (not the remote interface)

Notice that you don't say ".class" on the end. Just the class name.

```
File Edit Window Help Whuffle
%rmic MyRemoteImpl
```

RMIC generates two new classes for the helper objects.



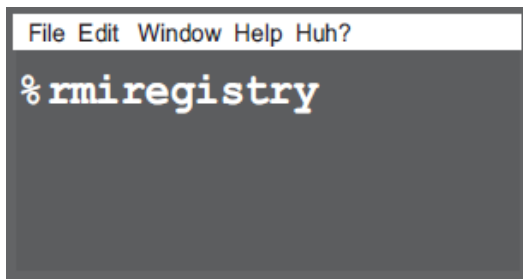
MyRemoteImpl_Stub.class



MyRemoteImpl_Skel.class

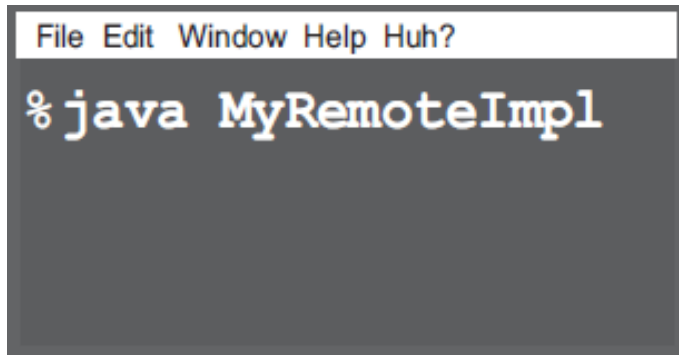
Step Four: Run rmiregistry

- Be sure you start it from a directory that has access to your classes.

A terminal window with a dark gray background and a light gray title bar. The title bar contains the text "File Edit Window Help Huh?". The main area of the terminal is dark gray and contains the text "%rmiregistry" in a light gray, monospaced font. The text is positioned at the top left of the terminal area, with a small gap before the command.

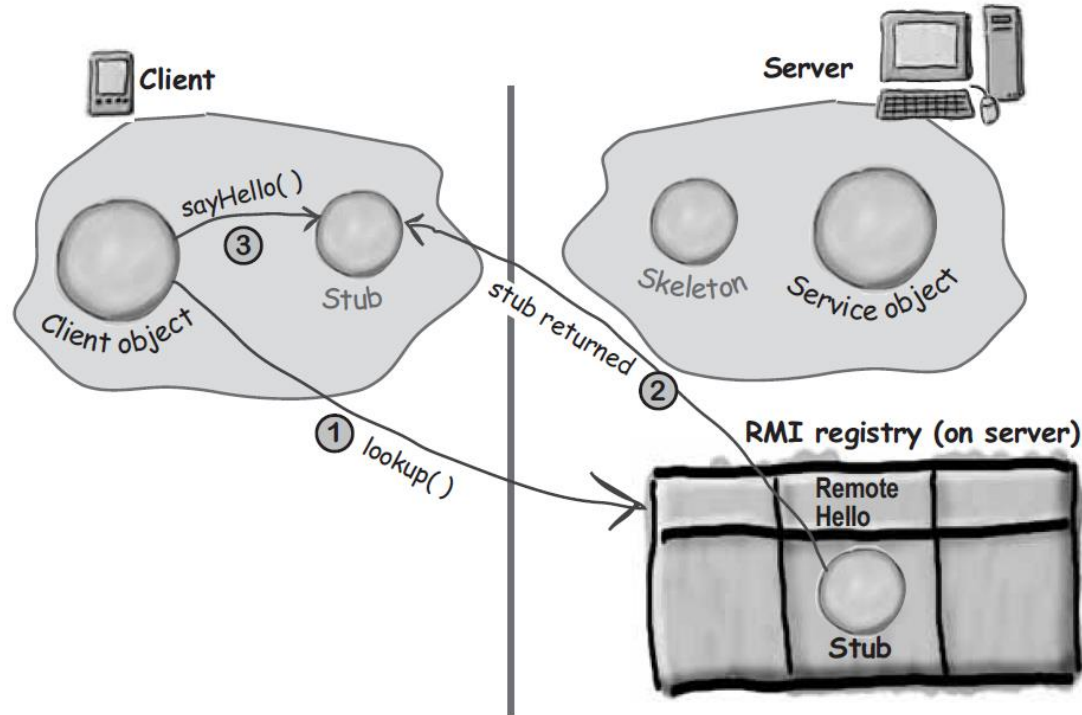
```
File Edit Window Help Huh?  
%rmiregistry
```

Step Five: Start the Service

A terminal window with a dark gray background and a white title bar. The title bar contains the text "File Edit Window Help Huh?". The terminal displays the command "% java MyRemoteImpl" in a light blue monospaced font.

```
File Edit Window Help Huh?  
% java MyRemoteImpl
```

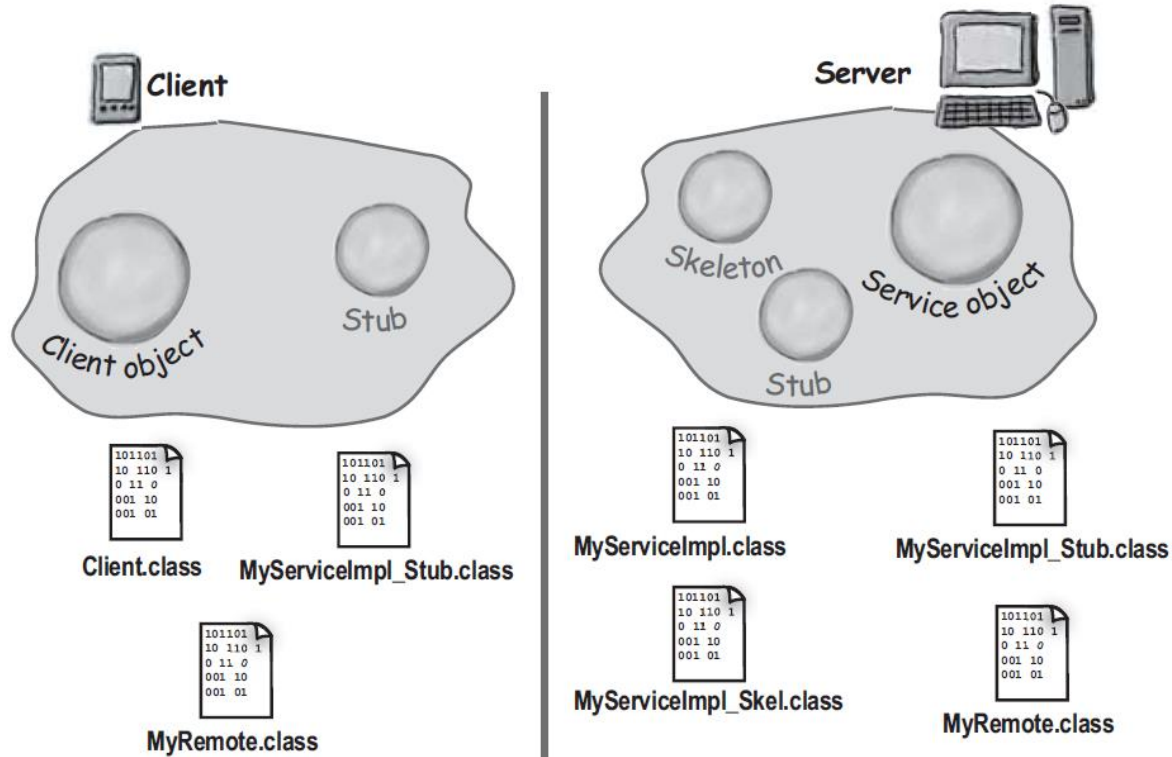
How Does Client Get the Stub Object?



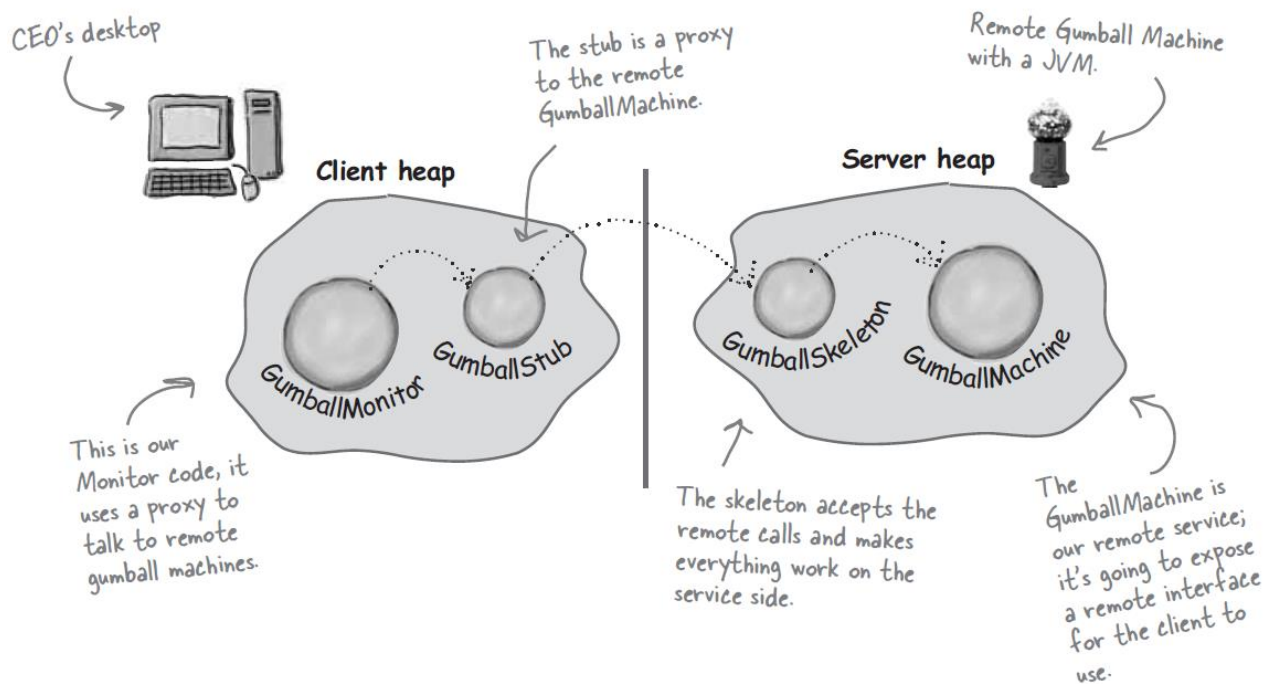
MyRemoteClient

```
import java.rmi.*;
public class MyRemoteClient {
    public static void main(String[] args) {
        new MyRemoteClient().go();
    }
    public void go() {
        try {
            MyRemote service = (MyRemote)
                Naming.lookup("rmi://127.0.0.1/RemoteHello");
            String s = service.sayHello();
            System.out.println(s);
        }
        catch (Exception ex) { ex.printStackTrace(); }
    }
}
```

After All, We Have These...



Distributed Gumball Machines



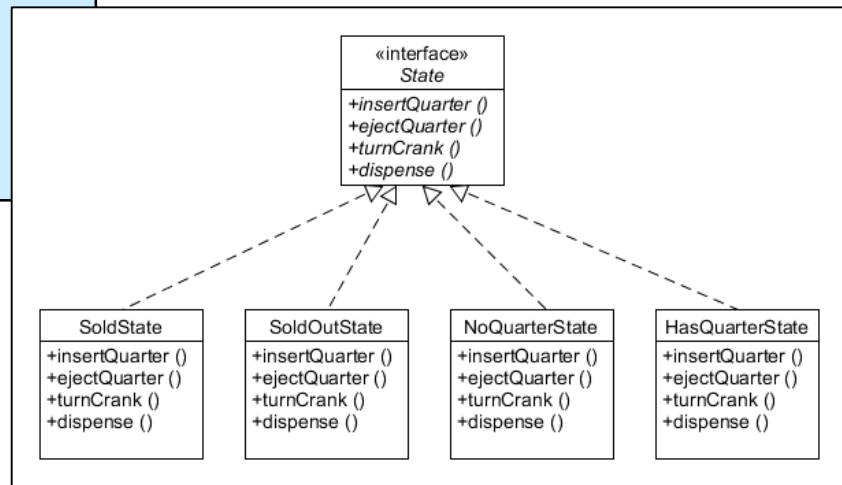
Code for Distributed Gumball Machines

Remote Interface

```
import java.rmi.*;
public interface GumballMachineRemote extends Remote {
    public int getCount() throws RemoteException;
    public String getLocation() throws RemoteException;
    public State getState() throws RemoteException;
}
```

State (Serializable object)

```
import java.io.*;
public interface State extends Serializable {
    public void insertQuarter();
    public void ejectQuarter();
    public void turnCrank();
    public void dispense();
}
```



Changes to the Concrete States

```
public class HasQuarterState implements State {
    transient GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }
    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }
    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }
    public void turnCrank() {
        System.out.println("You turned...");
        gumballMachine.setState(gumballMachine.getSoldState());
        gumballMachine.dispense();
    }
    public void dispense() {
        System.out.println("No Gumball dispensed");
    }
}
```

Code for Distributed Gumball Machines

Gumball Machine (Service Implementation)

```
import java.rmi.*;
import java.rmi.server.*;
public class GumballMachine
    extends UnicastRemoteObject implements GumballMachineRemote
{
    // instance variables
    public GumballMachine(String location, int numberGumballs)
        throws RemoteException {
        // constructor body
    }
    public int getCount() { return count; }
    public State getState() { return state; }
    public String getLocation() { return location; }
    public void insertQuarter() { state.insertQuarter(); }
    // other methods
}
```

Serialization basics

- Serialization
 - the process of **transforming** an **in-memory** object to a **byte stream**.
- Deserialization
 - **the inverse process** of reconstructing an object from a byte stream to the same state in which the object was previously serialized.
- For an object to be serializable
 - its class or some ancestor must **implement the *empty* Serializable interface**.
- An empty interface is called a *marker interface*.

Object graphs and transient fields

- If an object has references to other objects or arrays, the **entire object graph is serialized when the object is serialized**.
 - The object graph consists of the **object directly serialized and any other objects or arrays to which the object has** direct or indirect **references**.
- A field marked as **transient is not impacted by serialization**.
 - During deserialization, **transient** fields are restored to their *default* values (e.g., transient numeric fields are restored to zero).

Registering with the RMI Registry

```
import java.rmi.*;
public class GumballMachineTestDrive {
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("GumballMachine <name> <inventory>");
            System.exit(1);
        }
        GumballMachineRemote gumballMachine = null;
        int count;
        try {
            count = Integer.parseInt(args[1]);
            gumballMachine = new GumballMachine(args[0], count);
            Naming.rebind("//"+ args[0]+"/gumballmachine", gumballMachine);
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Gumball Monitor Client

```
import java.rmi.*;

public class GumballMonitor {
    GumballMachineRemote machine;
    public GumballMonitor(GumballMachineRemote machine) {
        this.machine = machine;
    }
    public void report() {
        try {
            System.out.println("Gumball Machine: " + machine.getLocation());
            System.out.println("Current inventory: " + machine.getCount()+ " gumballs");
            System.out.println("Current state: " + machine.getState());
        } catch (RemoteException e) { e.printStackTrace(); }
    }
}
```

Gumball Monitor TestDrive

```
import java.rmi.*;
public class GumballMonitorTestDrive {
    public static void main(String[] args) {
        String[] location = {
            "rmi://santafe.mightygumball.com/gumballmachine",
            "rmi://boulder.mightygumball.com/gumballmachine",
            "rmi://seattle.mightygumball.com/gumballmachine"};

        GumballMonitor[] monitor = new GumballMonitor[location.length];
        for (int i=0; i < location.length; i++) {
            try {
                GumballMachineRemote machine =
                    (GumballMachineRemote) Naming.lookup(location[i]);
                monitor[i] = new GumballMonitor(machine);
                System.out.println(monitor[i]);
            } catch (Exception e) { e.printStackTrace();}
        }

        for(int i=0; i < monitor.length; i++) monitor[i].report();
    }
}
```

Starting Gumball Machines

On each machine, run rmiregistry in the background or from a separate terminal window...

...and then run the GumballMachine, giving it a location and an initial gumball count.



The image shows three overlapping terminal windows, each with a menu bar (File, Edit, Window, Help, Huh?) and a dark background. The first window shows the commands `% rmiregistry &` and `% java GumballMachine santafe.mightygumball.com 100`. The second window shows `% rmiregistry &` and `% java GumballMachine boulder.mightygumball.com 100`. The third window shows `% rmiregistry &` and `% java GumballMachine seattle.mightygumball.com 250`. A handwritten note "popular machine!" with a small arrow points to the "seattle" location in the third window. Arrows from the explanatory text point to the `rmiregistry &` commands in the first and second windows.

```
File Edit Window Help Huh?  
% rmiregistry &  
% java GumballMachine santafe.mightygumball.com 100  
  
File Edit Window Help Huh?  
% rmiregistry &  
% java GumballMachine boulder.mightygumball.com 100  
  
File Edit Window Help Huh?  
% rmiregistry &  
% java GumballMachine seattle.mightygumball.com 250  
popular machine! ↗
```

```
File Edit Window Help GumballsAndBeyond
% java GumballMonitor
Gumball Machine: santafe.mightygumball.com
Current inventory: 99 gumballs
Current state: waiting for quarter

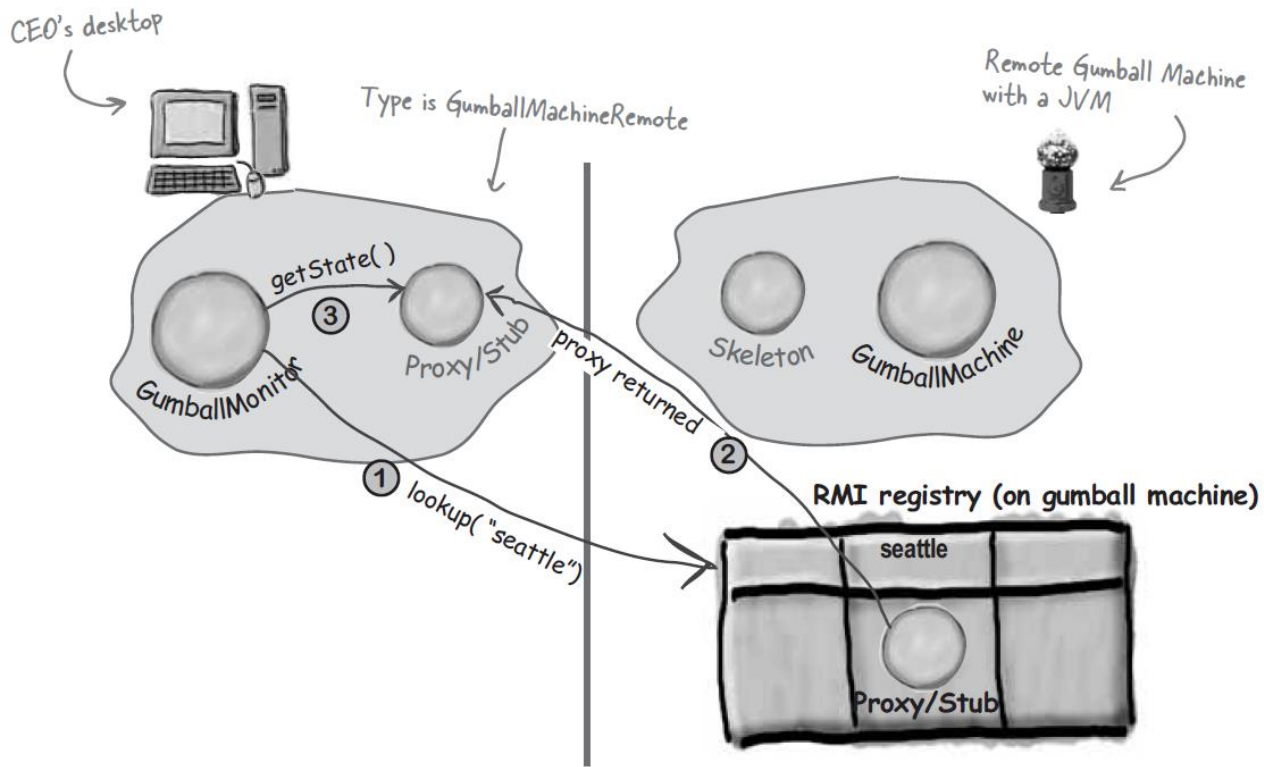
Gumball Machine: boulder.mightygumball.com
Current inventory: 44 gumballs
Current state: waiting for turn of crank

Gumball Machine: seattle.mightygumball.com
Current inventory: 187 gumballs
Current state: waiting for quarter
%
```

The monitor iterates over each remote machine and calls its getLocation(), getCount() and getState() methods.

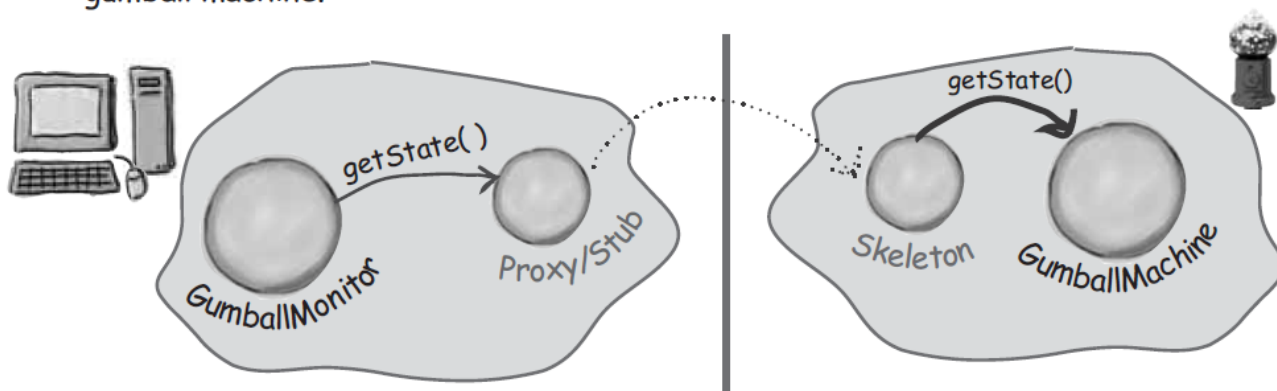
This is amazing; it's going to revolutionize my business and blow away the competition!

Behind the Scenes



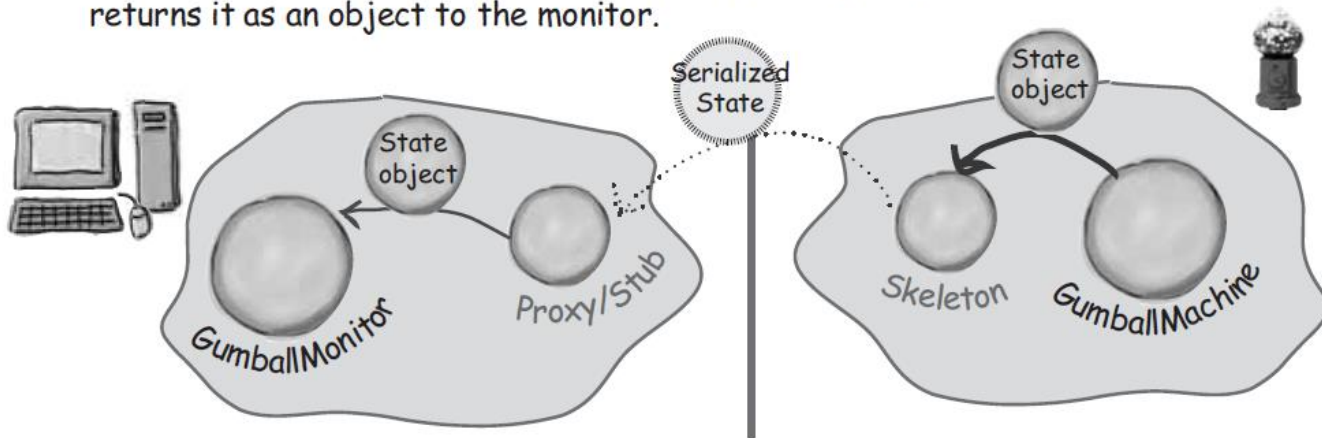
Behind the Scenes

- 2 `getState()` is called on the proxy, which forwards the call to the remote service. The skeleton receives the request and then forwards it to the gumball machine.



Behind the Scenes

- 3 GumballMachine returns the state to the skeleton, which serializes it and transfers it back over the wire to the proxy. The proxy deserializes it and returns it as an object to the monitor.



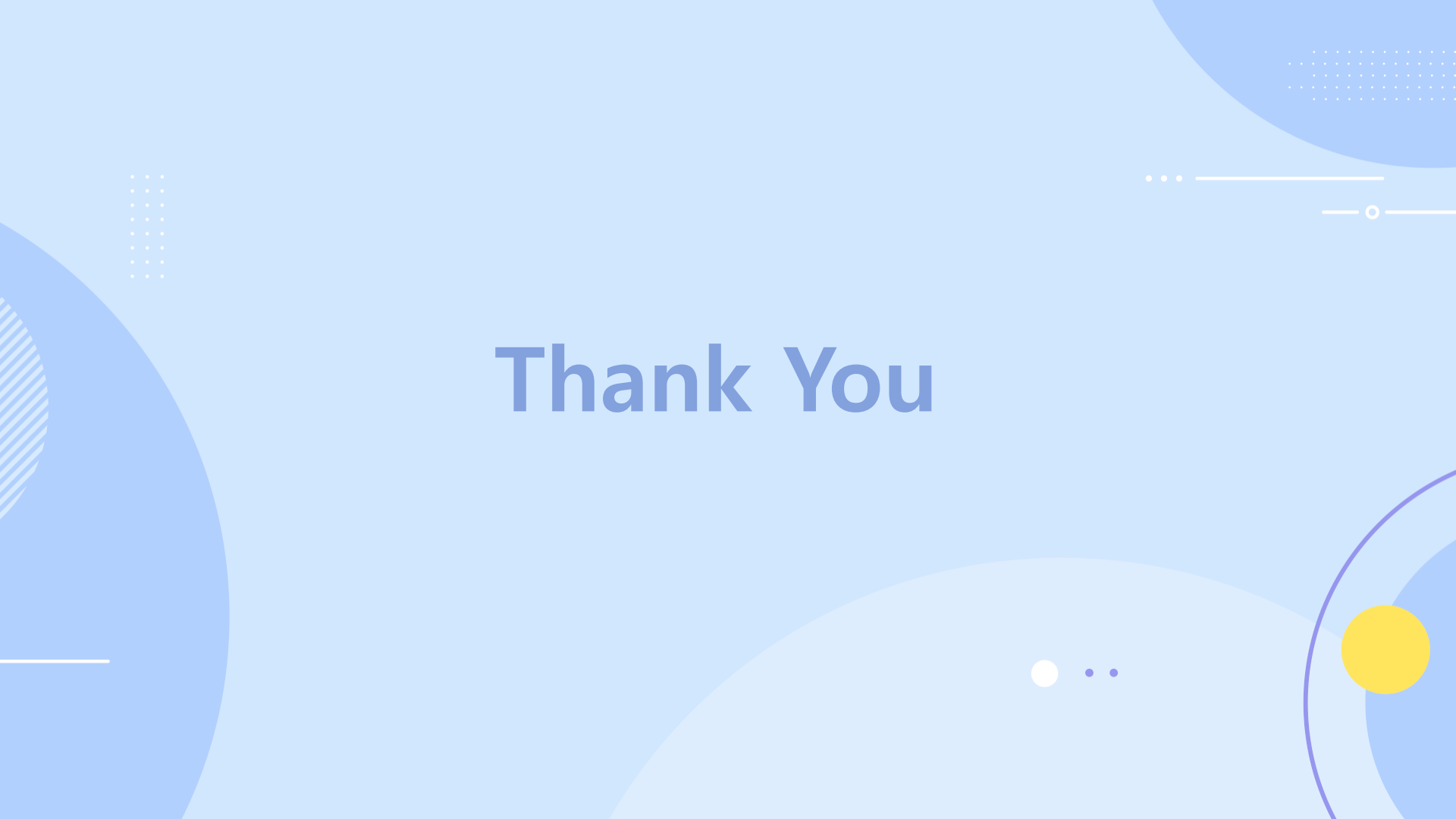
Review - Related Patterns

- w.r.t. Interface

- **Adapter** provides a different interface to its subject
- **Proxy** provides the same interface
- **Decorator** provides an enhanced interface

- w.r.t. Structure

- **Decorator and Proxy have similar structures**
- Both describe how to provide a level of indirection to another object, and the implementations keep a reference to the object to which they forward requests



Thank You

A vertical blue sidebar on the left side of the slide. It contains several decorative elements: a large blue circle at the top left with a white dotted pattern; a smaller blue circle at the top right with a white diagonal line pattern; a vertical white line with a small white circle in the middle and three small white dots at the bottom; a large blue circle at the bottom left with a white dotted pattern; a smaller blue circle at the bottom right with a white diagonal line pattern; and a small red circle at the bottom left.

05

Visitor Pattern

Visitor Pattern

- Purpose
 - Allowing one or more operations to be applied to a set of objects at runtime
 - Decoupling the operations from the object structure (= the set of objects)
- Use when
 - An object structure must have many unrelated operations performed upon it
 - The object structure can't change but operations on it can
 - Operations should be able to operate on multiple object structures that implement the same interface sets

Example : Abstract Syntax Tree

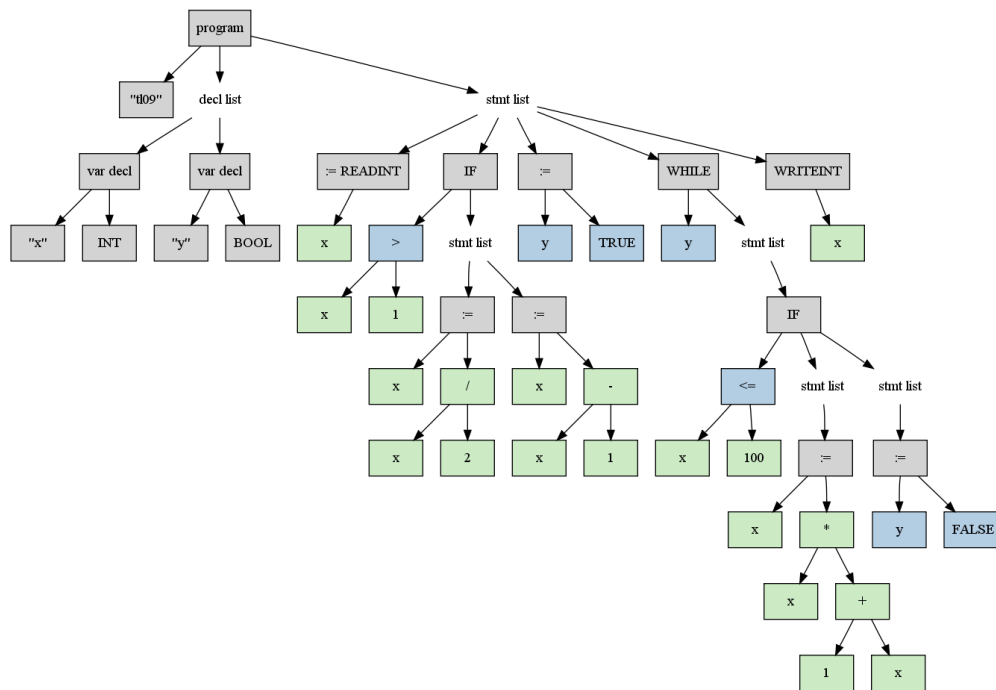
- Consider a compiler with abstract syntax trees
- Compiler may need to define operations for

Type-Checking

Code Optimization

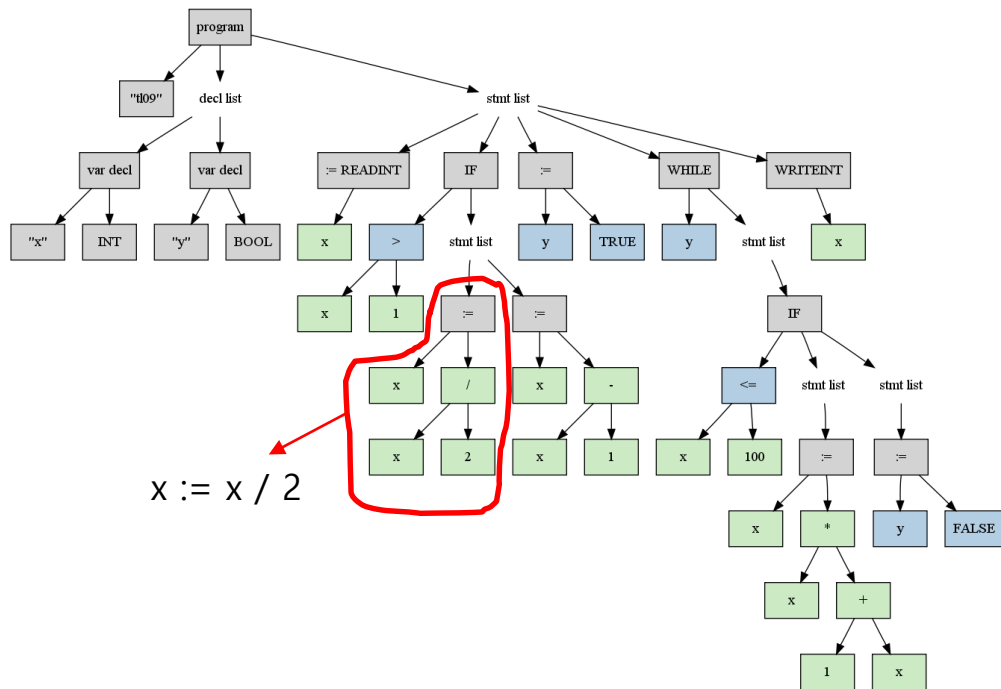
Flow Analysis

... and more!



Implementing Type-Checking Operation

- Seemingly not an issue
 - Just **implement at each AST node** -> but there are **additional operations**!

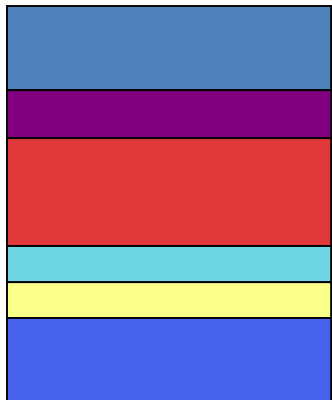


More operation types are expected

- One class for every single thing that needs to be performed.
- Distributing operations amongst various node classes is hard to understand, maintain and change!
 - Adding a new operation would be painful

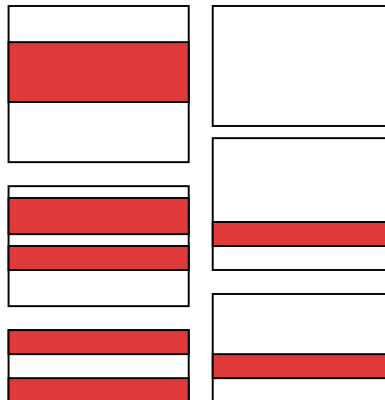
code tangling:

one module with many concerns



code scattering:

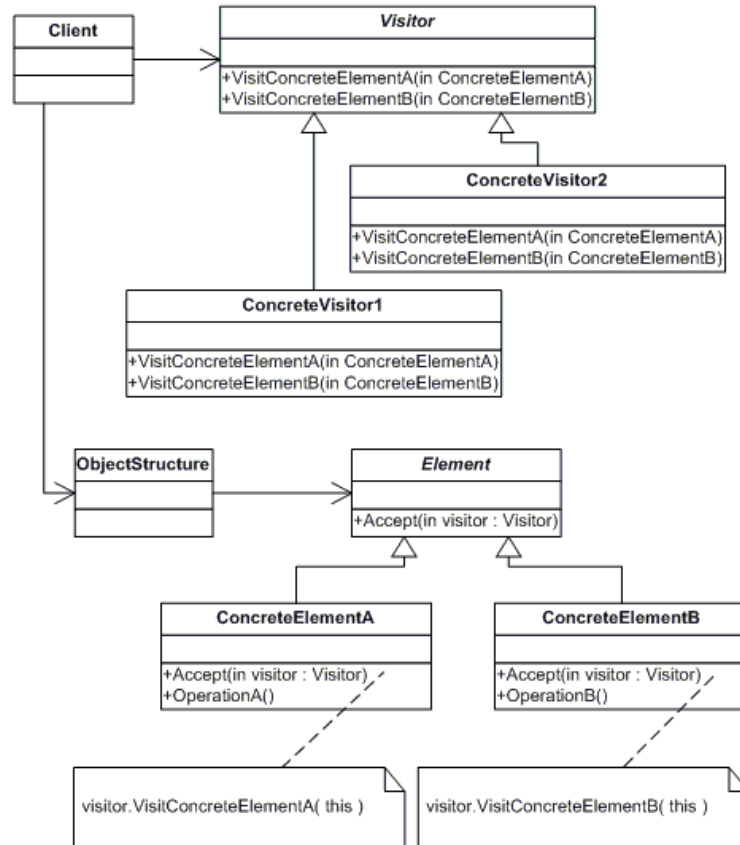
one concern scatted amongst many modules



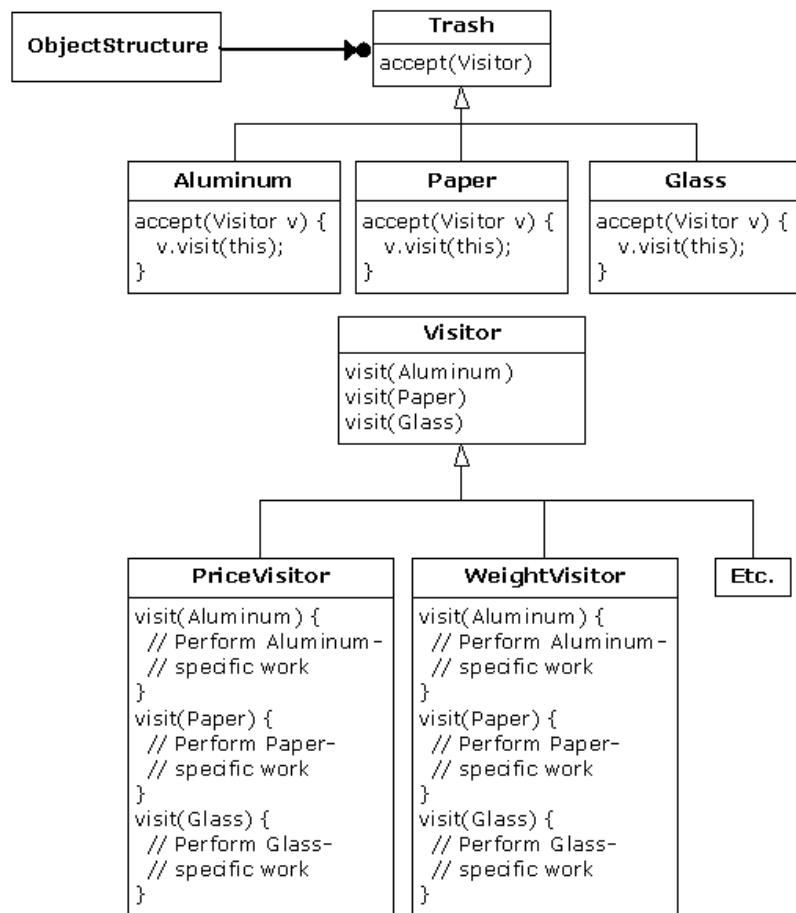
Motivations and Benefits

- Motivations
 - An object structure contains many classes of objects
 - Many distinct and unrelated operations on these objects
 - We want to **avoid "polluting" their classes**
 - **Classes defining the object structure rarely change, but operations change frequently**
- Benefits
 - Makes **adding new** operations **easy**
 - **Gathers related** operations and **separates unrelated** operations

Visitor Pattern in class diagram



Visitor Example

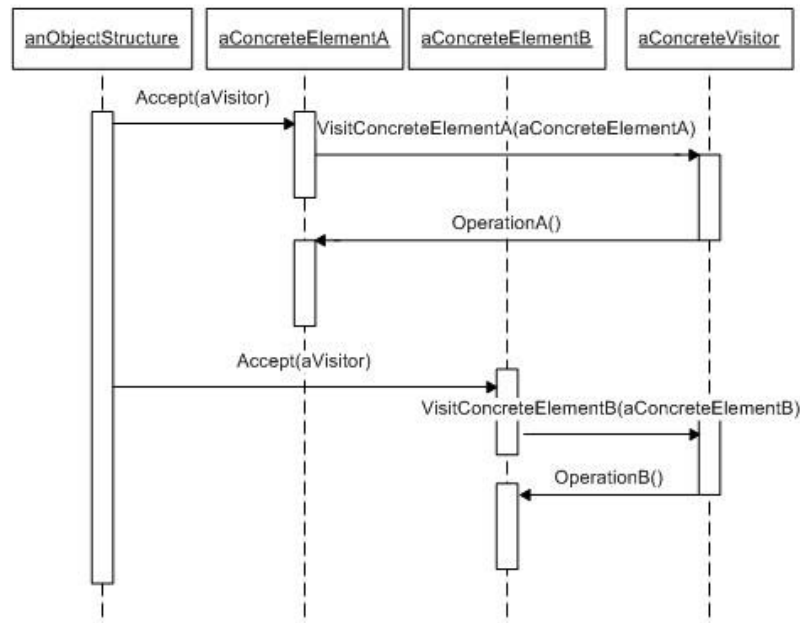


Participants

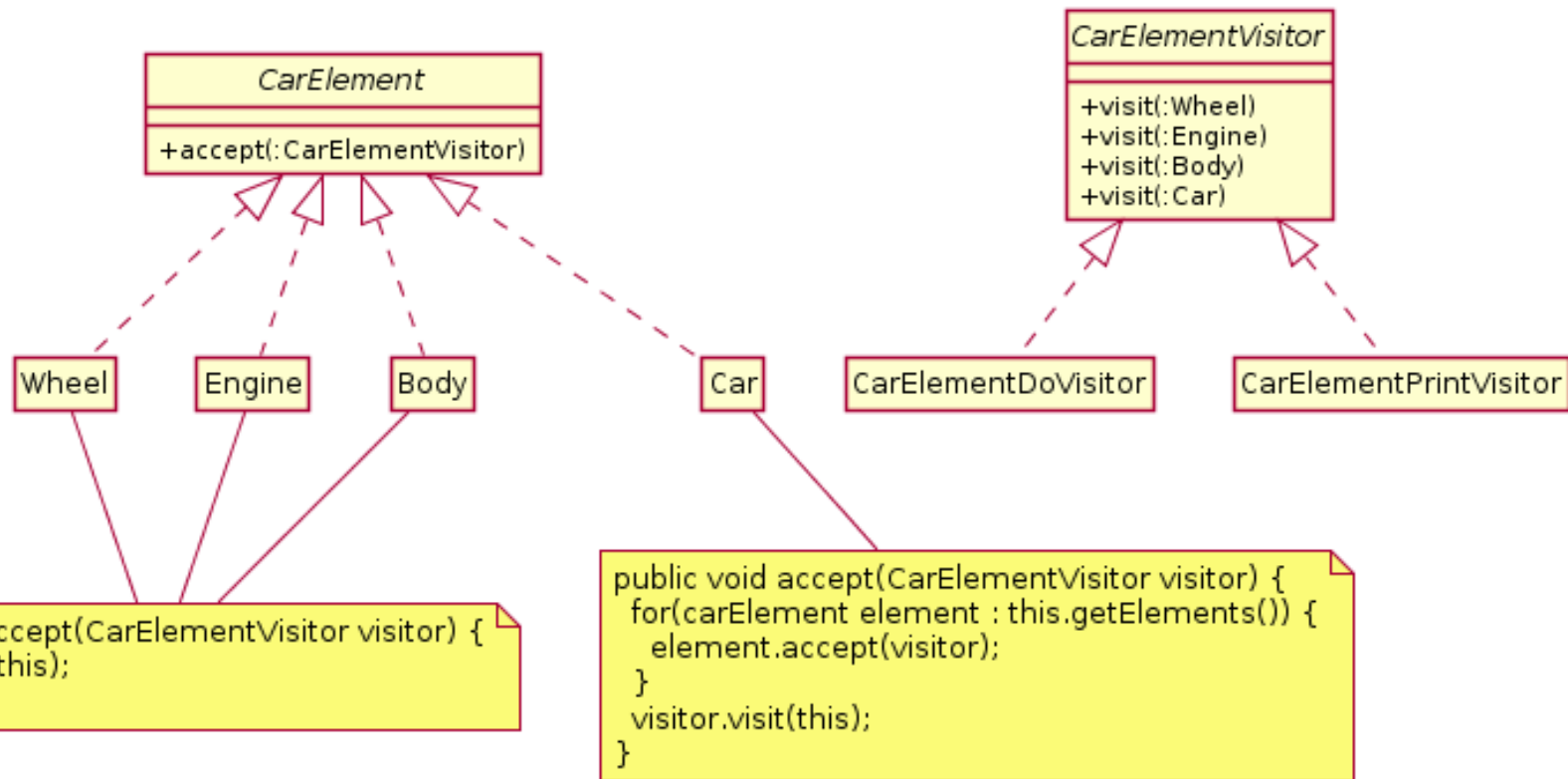
- Visitor
 - declares a visit operation for each class of ConcreteElement in the object structure
- Concrete Visitor (PriceVisitor, WeightVisitor)
 - implements each operation declared by Visitor
- Element (Trash)
 - defines an Accept operation that takes a visitor as an argument
- Concrete Element (Aluminum, Paper, Glass)
 - implements an Accept operation that takes a visitor as an argument
- Object Structure
 - can enumerate its elements
 - may provide a high-level interface to allow the visitor to visit its elements
 - may be a composite or a collection like a set or list

Collaborations

- A client must create a ConcreteVisitor object and then traverse the object structure, visiting each element with the visitor
- When the element is visited, it calls the Visitor operation that corresponds to its class.
- The element supplies itself as an argument to this operation to let the visitor access its state, if necessary



Another Example of Visitor Pattern



Visitor Interface and Visitor Invocation

```
interface ICarElementVisitor {  
    void visit(Wheel wheel);  
    void visit(Engine engine);  
    void visit(Body body);  
    void visit(Car car);  
}  
  
interface ICarElement {  
    void accept(ICarElementVisitor visitor);  
}  
  
public class VisitorDemo {  
    public static void main(String[] args) {  
        ICarElement car = new Car();  
        car.accept(new CarElementPrintVisitor());  
        car.accept(new CarElementDoVisitor());  
    }  
}
```

Concrete Elements

```
class Wheel implements ICarElement {
    private String name;
    public Wheel(String name) { this.name = name; }
    public String getName() { return this.name; }
    public void accept(ICarElementVisitor visitor) {
        visitor.visit(this);
    }
}

class Engine implements ICarElement {
    public void accept(ICarElementVisitor visitor) {
        visitor.visit(this);
    }
}

class Body implements ICarElement {
    public void accept(ICarElementVisitor visitor) {
        visitor.visit(this);
    }
}
```

Object Structure and Concrete Element

```
class Car implements ICarElement {  
    ICarElement[] elements;  
    public Car() {  
        this.elements = new ICarElement[] {  
            new Wheel("front left"), new Wheel("front right"),  
            new Wheel("back left") , new Wheel("back right"),  
            new Body(),  
            new Engine() };  
    }  
    public void accept(ICarElementVisitor visitor) {  
        for(ICarElement elem : elements)  
            elem.accept(visitor);  
        visitor.visit(this);  
    }  
}
```

Concrete Visitor 1

```
class CarElementPrintVisitor implements ICarElementVisitor {  
    public void visit(Wheel wheel) {  
        System.out.println("Visiting " + wheel.getName() + " wheel");  
    }  
    public void visit(Engine engine) {  
        System.out.println("Visiting engine");  
    }  
    public void visit(Body body) {  
        System.out.println("Visiting body");  
    }  
    public void visit(Car car) {  
        System.out.println("Visiting car");  
    }  
}
```

Concrete Visitor 2

```
class CarElementDoVisitor implements ICarElementVisitor {  
    public void visit(Wheel wheel) {  
        System.out.println("Kicking my " + wheel.getName() + " wheel");  
    }  
    public void visit(Engine engine) {  
        System.out.println("Starting my engine");  
    }  
    public void visit(Body body) {  
        System.out.println("Moving my body");  
    }  
    public void visit(Car car) {  
        System.out.println("Starting my car");  
    }  
}
```

Output

Visiting front left wheel

Visiting front right wheel

Visiting back left wheel

Visiting back right wheel

Visiting body

Visiting engine

Visiting car

Kicking my front left wheel

Kicking my front right wheel

Kicking my back left wheel

Kicking my back right wheel

Moving my body

Starting my engine

Starting my car

Flow of the program

```
car.accept(new CarElementPrintVisitor());
```

Car::accept()

```
for(ICarElement elem : elements)
    elem.accept(visitor);
```

Engine::accept()

```
public void accept(ICarElementVisitor visitor) {
    visitor.visit(this);
}
```

CarElementVisitor::visit()

```
class CarElementPrintVisitor implements ICarElementVisitor {
    ...
    public void visit(Engine engine) {
        System.out.println("Visiting engine");
    }
    ...
}
```

Question: Do We Really Need to Repeat These?

```
class Wheel implements ICarElement {  
    public void accept(ICarElementVisitor visitor) {  
        visitor.visit(this);  
    }  
}  
  
class Engine implements ICarElement {  
    public void accept(ICarElementVisitor visitor) {  
        visitor.visit(this);  
    }  
}  
  
class Body implements ICarElement {  
    public void accept(ICarElementVisitor visitor) {  
        visitor.visit(this);  
    }  
}
```

Removing duplicate accept methods (Attempt)

- Change ICarElement **interface** to ICarElement **abstract class**. Implement accept method there.
- Replace "**implements**" with "**extends**" on each concrete element
- Remove "**accept**" method in each concrete element

```
interface ICarElement {  
    void accept(ICarElementVisitor visitor);  
}
```



```
abstract class ICarElement {  
    void accept(ICarElementVisitor visitor) {  
        visitor.visit(this);  
    }  
}
```

```
class Wheel implements ICarElement {  
    public void accept(ICarElementVisitor  
        visitor)  
    {  
        visitor.visit(this);  
    }  
}
```



```
class Wheel extends ICarElement {  
    public void accept(ICarElementVisitor  
    visitor)  
    {  
        visitor.visit(this);  
    }  
}
```

Removing duplicate accept methods (Attempt)

You need to add dummy visit(ICarElement) into each concrete visitor

```
interface ICarElementVisitor {  
    void visit(Wheel wheel);  
    void visit(Engine engine);  
    void visit(Body body);  
    void visit(Car car);  
}
```



```
interface ICarElementVisitor {  
    void visit(Wheel wheel);  
    void visit(Engine engine);  
    void visit(Body body);  
    void visit(Car car);  
    void visit(ICarElement dummy);  
}
```

```
class CarElementPrintVisitor implements  
ICarElementVisitor {  
    public void visit(Wheel wheel) {  
        System.out.println("Visiting " +  
wheel.getName() + " wheel");  
    }  
    public void visit(Engine engine) {  
        System.out.println("Visiting engine");  
    }  
    public void visit(Body body) {  
        System.out.println("Visiting body");  
    }  
    public void visit(Car car) {  
        System.out.println("Visiting car");  
    }  
}
```



```
class CarElementPrintVisitor implements  
ICarElementVisitor {  
    public void visit(Wheel wheel) {  
        System.out.println("Visiting " +  
wheel.getName() + " wheel");  
    }  
    public void visit(Engine engine) {  
        System.out.println("Visiting engine");  
    }  
    public void visit(Body body) {  
        System.out.println("Visiting body");  
    }  
    public void visit(Car car) {  
        System.out.println("Visiting car");  
    }  
    public void visit(ICarElement dummy) {  
        System.out.println("!!! Dummy !!!");  
    }  
}
```

Output (Attempt Failed)

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

Visiting car

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

!!! Dummy !!!

Starting my car

Method Overloading - Static Polymorphism

```
class X
{
    void methodA(int num)
    {
        System.out.println ("methodA:" + num);
    }
    void methodA(int num1, int num2)
    {
        System.out.println ("methodA:" + num1 + "," + num2);
    }
    double methodA(double num) {
        System.out.println("methodA:" + num);
        return num;
    }
}

public class test
{
    public static void main (String args [])
    {
        X Obj = new X();
        double result;
        Obj.methodA(20);
        Obj.methodA(20, 30);
        result = Obj.methodA(5.5);
        System.out.println("Answer is:" + result);
    }
}
```

methodA:20
methodA:20,30
methodA:5.5
Answer is:5.5

Method Overriding - Dynamic Polymorphism

```
public class X
{
    public void methodA() //Base class method
    {
        System.out.println ("hello, I'm methodA of class X");
    }
}

public class Y extends X
{
    public void methodA() //Derived Class method
    {
        System.out.println ("hello, I'm methodA of class Y");
    }
}

public class Z
{
    public static void main (String args []) {
        X obj1 = new X(); // Reference and object X
        X obj2 = new Y(); // X reference but Y object
        obj1.methodA();
        obj2.methodA();
    }
}
```

```
hello, I'm methodA of class X
hello, I'm methodA of class Y
```

Method Overloading

- Method overloading is done at compile time
 - name mangling: the internal name of the method has the argument's type encoded in it
 - Different from method overriding (run-time polymorphism)

- Example

- method foo(int) → __foo_i
- method foo(double) → __foo_d

```
class Wheel implements ICarElement {  
    public void accept(ICarElementVisitor visitor)  
    {  
        visitor.visit(this);  
    }  
}
```

- Double Dispatch
 - special form of [multiple dispatch](#)
 - a mechanism that dispatches a method call to different concrete methods depending on the runtime types of two objects involved in the call
- Single Dispatch
 - In most [object-oriented](#) systems, the concrete method that is called from a method call in the code depends on the dynamic type of a single object and therefore they are known as single dispatch calls or simply [virtual function](#)(method) calls

Java and C++ do not support double dispatch

- The following implementation does not work!

```
1 class Car implements ICarElement {  
2     ICarElement[] elements;  
3     public void applyVisitor(ICarElementVisitor visitor) {  
4         for(ICarElement elem : elements)  
5             visitor.visit(elem); // WON'T WORK AS EXPECTED!  
6         visitor.visit(this);  
7     }  
8 }
```

Visitor Pattern and Double Dispatch

- Visitor pattern requires a programming language that supports double dispatch
 - Consider two objects: "element" and "visitor"

```
1 class Car implements ICarElement {
2     ICarElement[] elements;
3     public void accept(ICarElementVisitor visitor) {
4         for(ICarElement elem : elements)
5             // visitor.visit(elem);
6             elem.accept(visitor);
7         visitor.visit(this);
8     }
9 }
```

```
1 class Engine implements ICarElement {
2     public void accept(ICarElementVisitor visitor) {
3         visitor.visit(this);
4     }
5 }
```

- An **element** has an **accept method** that can take the visitor as an argument
- The **accept method** calls a **visit method of the visitor**; the element passes itself as an argument to the visit method

How Double-dispatch is implemented in Visitor Pattern

- When the accept method is called, its implementation is chosen based on both:
 - The dynamic type of the element
 - The static type of the visitor
- When the associated visit method is called, its implementation is chosen based on both:
 - The dynamic type of the visitor
 - The static type of the element as known from within the implementation of the accept method, which is the same as the dynamic type of the element. (As a bonus, if the visitor can't handle an argument of the given element's type, then the compiler will catch the error)

```
for(ICarElement elem : elements)  
    elem.accept(visitor);
```

```
public void accept(ICarElementVisitor visitor) {  
    visitor.visit(this);  
}
```

How Double-dispatch is implemented in Visitor Pattern

- Consequently, the implementation of the visit method is chosen based on both:
 - The dynamic type of the element
 - The dynamic type of the visitor
- This effectively implements double dispatch
 - Common Lisp language's object system supports multiple dispatch (not just single dispatch), and implementing the visitor pattern in Common Lisp is trivial

Review - Benefits

- Visitor makes adding new operations easy
 - You can define a new operation simply by adding a new visitor
 - In contrast, if you spread functionality over many classes, then you must change each class to define a new operation
- A visitor gathers related operations and separates unrelated ones
 - Related behavior is not spread over the classes defining the object structure; it's localized in a visitor
 - Unrelated sets of behavior are partitioned in their own visitor classes

Review - Liabilities

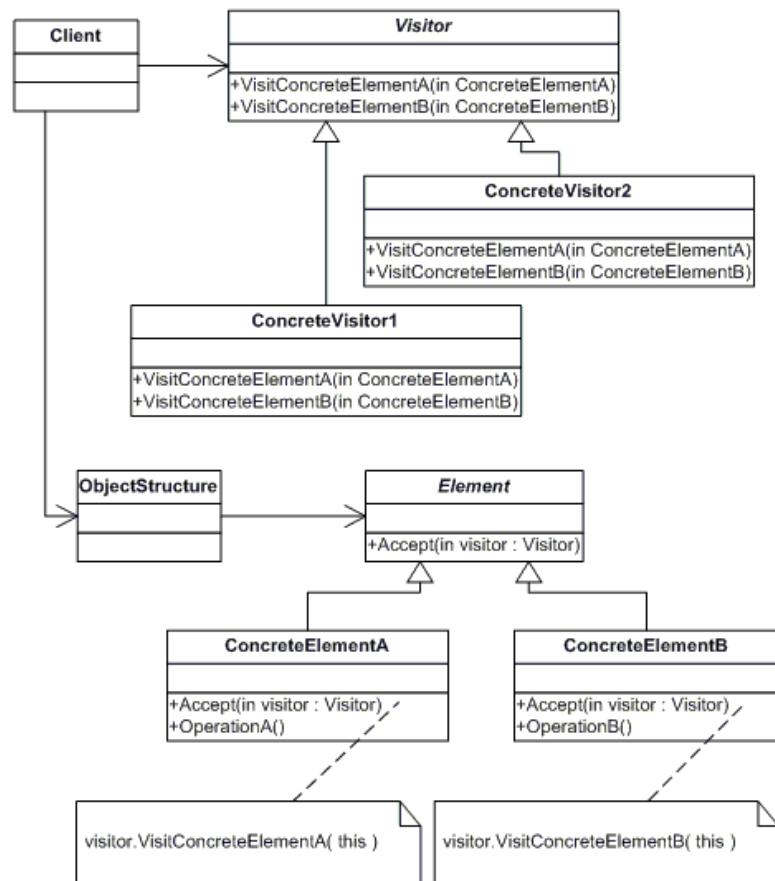
- Adding new ConcreteElement classes is hard
 - The Visitor pattern makes it hard to add new subclasses of Element
 - Each new ConcreteElement gives rise to a new abstract operation on Visitor and a corresponding implementation in every ConcreteVisitor class
- Breaking encapsulation
 - Visitor's approach assumes that the ConcreteElement interface is powerful enough to let visitors do their job
 - The pattern often forces you to provide public operations that access an element's internal state, which may compromise its encapsulation

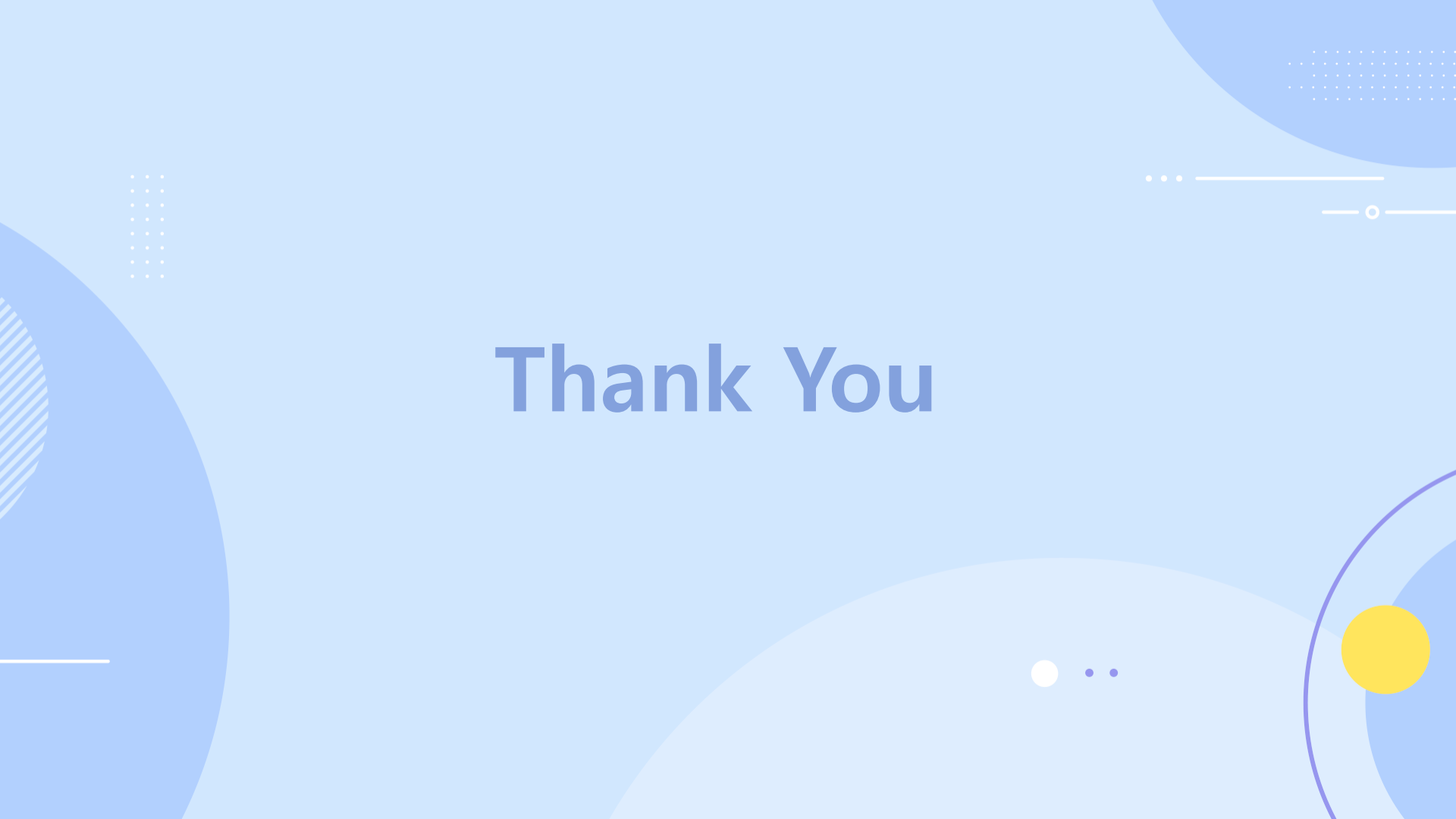
Review - Implementation of Traversal

- Who is responsible for traversing the object structure?
 - A visitor must visit each element of the object structure. How does it get there?
 - We can put responsibility for traversal in any of three places:
 - In the object structure
 - in the visitor
 - or in a separate Iterator object
 - Having the traversal code in the visitor is the least preferred option, as it forces you to repeat the code in every ConcreteVisitor for each ConcreteElement;
 - so you would only follow this approach for a particularly complex traversal

Review - Summary

- The Visitor Design Pattern
 - (+) makes adding new operations easy
 - (+) gathers related operations and separates unrelated ones
 - (-) adding new ConcreteElement classes is hard
 - (-) Breaks encapsulation
- Implementation Issues
 - Who is responsible for traversing the object structure?





Thank You

The left side of the slide features a light blue background with various geometric elements. At the top left, there is a dark blue circle with a white dotted pattern. Below it, a vertical white line with a small circle in the middle and three dots at the bottom is visible. Further down, another dark blue circle with a white dotted pattern is present. At the bottom left, there is a white semi-circle with a pink circle on its edge and three dots below it. A large, dark blue circle with a white dotted pattern is also visible in the lower right quadrant of the blue area.

06

Facade Pattern

Facade Pattern

- **Purpose**

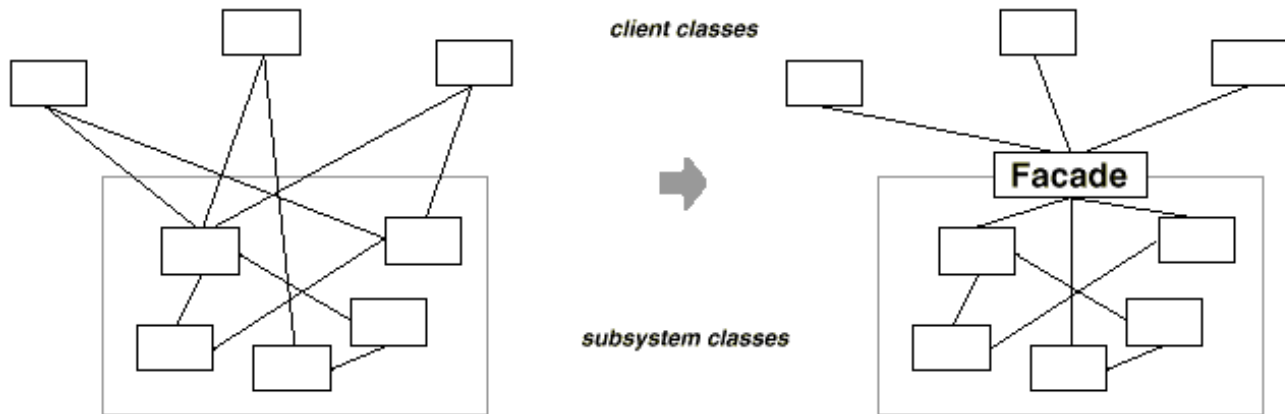
- Supplies a single interface to a set of interfaces within a system.

- **Use When**

- A simple interface is needed to provide access to a complex system.
- There are many dependencies between system implementations and clients.
- Systems and subsystems should be layered.

The Facade Pattern

- Provides a unified interface to a set of interfaces in a subsystem. It defines a higher-level interface that makes a subsystem easier to use



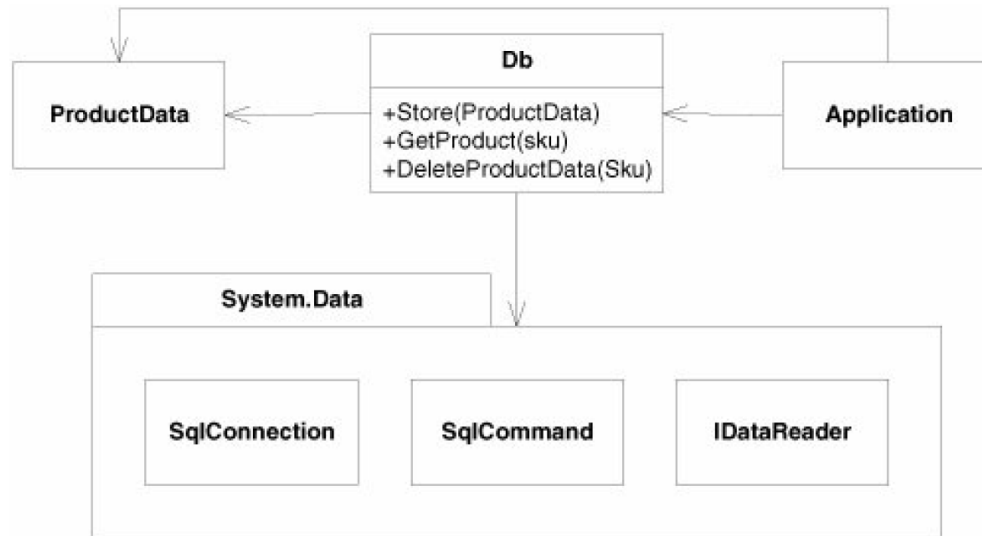
Motivation

- In typical OO Design,
 - Structuring a system into subsystems helps reduce complexity
 - Subsystems are groups of classes, or groups of classes and other subsystems
 - May produces many minimal classes
- Problems
 - Class/Subsystem interface can become quite complex
 - Too many options to use!
 - A new-comer cannot figure out where to begin
- Solution
 - Facade object provides a single, simplified interface to the more general facilities of a subsystem

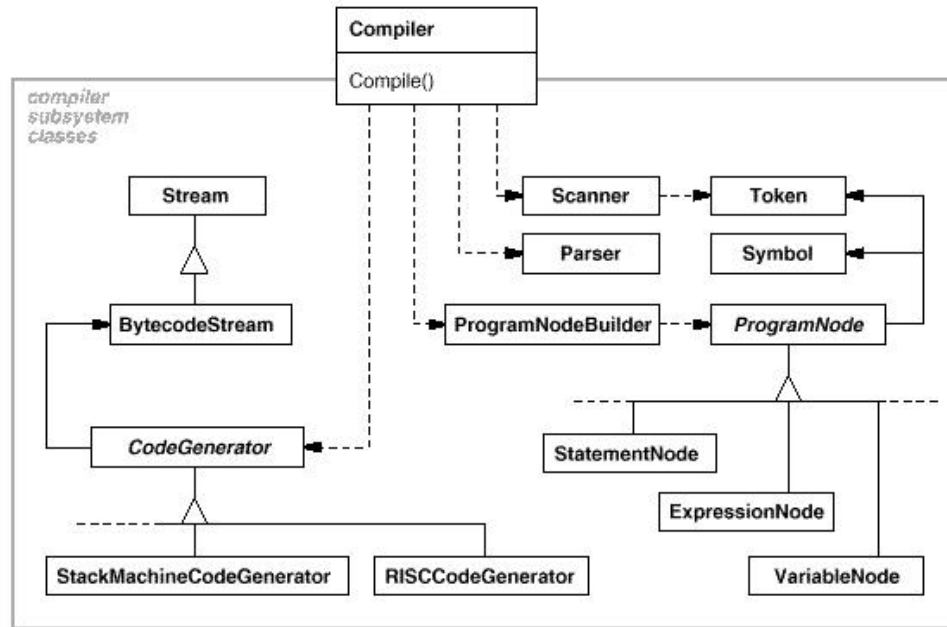
Benefits

- Hides the implementation of the subsystem from clients
 - makes the subsystem easier to use
- Promotes weak coupling between the subsystem and its clients
 - Allows changing the classes comprising the subsystem without affecting the clients
- **Does not prevent sophisticated clients from accessing the underlying classes**
- Notice: **Facade does not add any functionality**, it just **simplifies interfaces**

Example



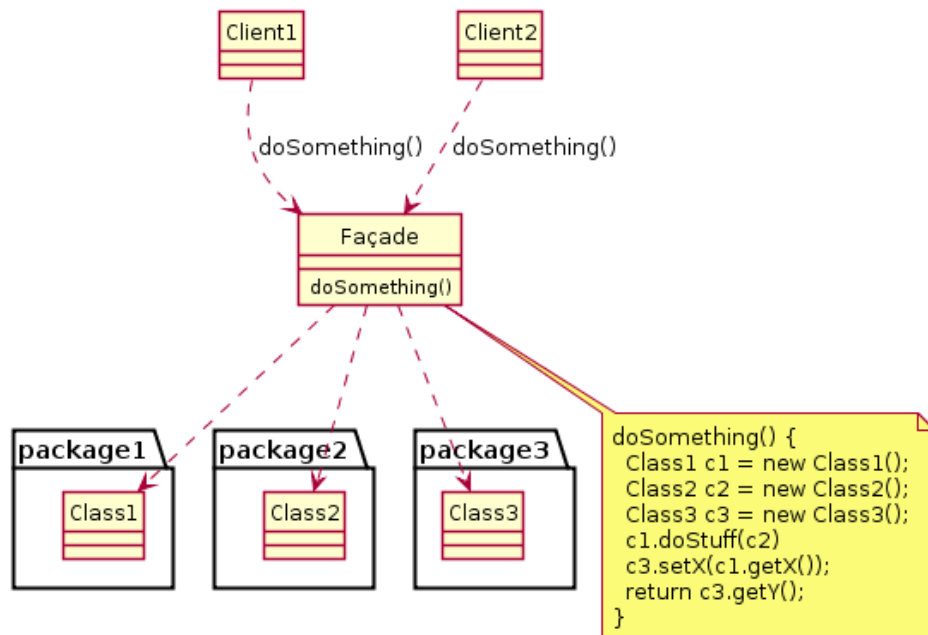
Example



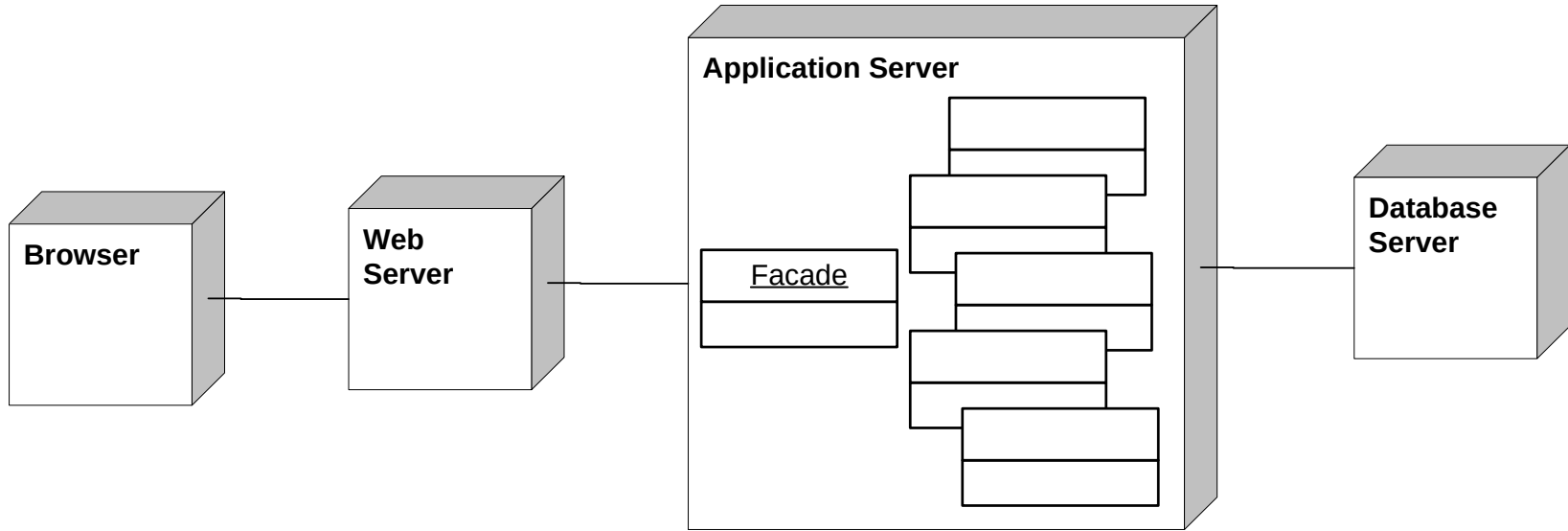
Example Code

```
class Scanner { //... };
class Parser { //... };
...
class CodeGenerator { //... };
class Compiler { // facade class
public:
    Compiler();
    virtual void Compile(istream&, BytecodeStream&);
};
void Compiler::Compile(istream& input, BytecodeStream& output) {
    Scanner scanner(input);
    ProgramNodeBuilder builder;
    Parser parser;
    parser.Parse(scanner, builder);
    RISCCodeGenerator generator(output);
    ProgramNode* parseTree = builder.GetRootNode();
    parseTree->Traverse(generator);
}
```

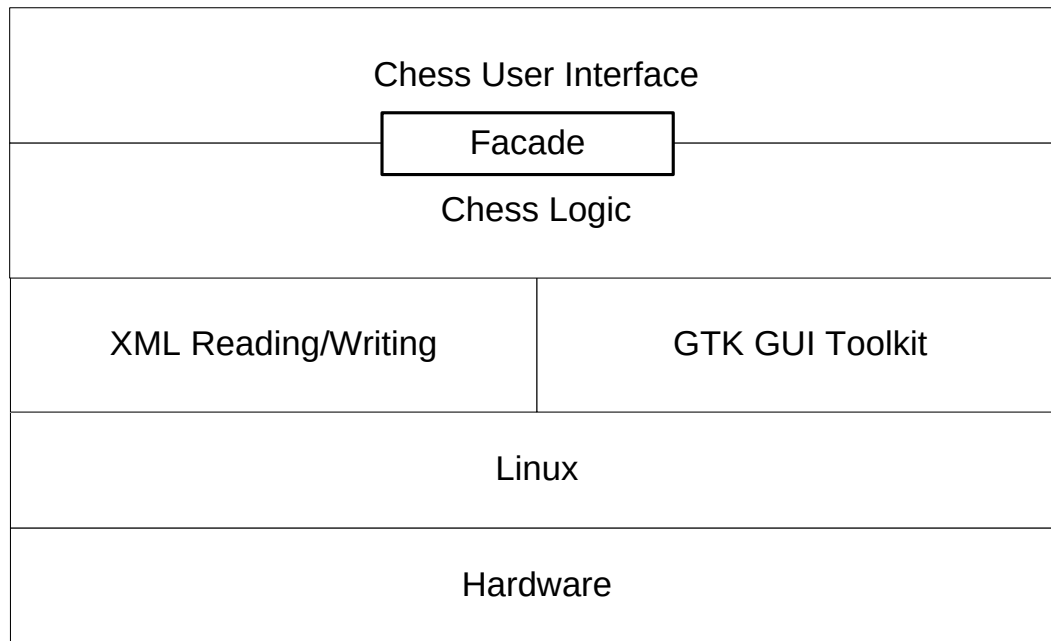
Facade Example



Known Uses: Web Applications



Facade in Layered Architecture



Comparing the Facade with the Adapter Pattern

- Are there existing classes?
- Is there an interface we must design to?
- Is a simpler interface needed?

A decorative vertical sidebar on the left side of the slide. It features a light blue background with various geometric elements: a dark blue circle with a white dotted pattern in the upper left, a white vertical line with a small circle in the middle and three dots at the bottom, a dark blue circle with a white dotted pattern in the lower left, and a large dark blue circle with a white dotted pattern in the lower right. A small red circle is also visible near the bottom left.

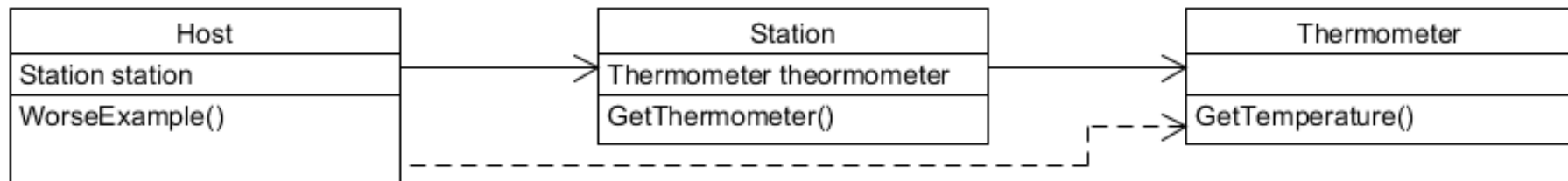
The Principle of Least Knowledge

The Principle of Least Knowledge (Law of Demeter)

- Talk only to your immediate friends
 - When you design a system, you should be careful of the number of classes it interacts with and also how it comes to interact with those classes
 - A method `m` of an object `o` may only invoke the methods of the following kinds of objects
 - `o` itself
 - `m`'s parameters
 - any objects created/instantiated within `m`
 - `o`'s direct component objects
 - a global variable, accessible by `o`, in the scope of `m`
 - i.e., "use only one dot"
 - `a.b.Method()` breaks the law where `a.Method()` does not

Example

```
public float worseExample() {  
    Thermometer thermometer = station.GetThermometer();  
    return thermometer.GetTemperature();  
}  
  
public float BetterExample() {  
    return station.GetTemperature();  
}
```



Example

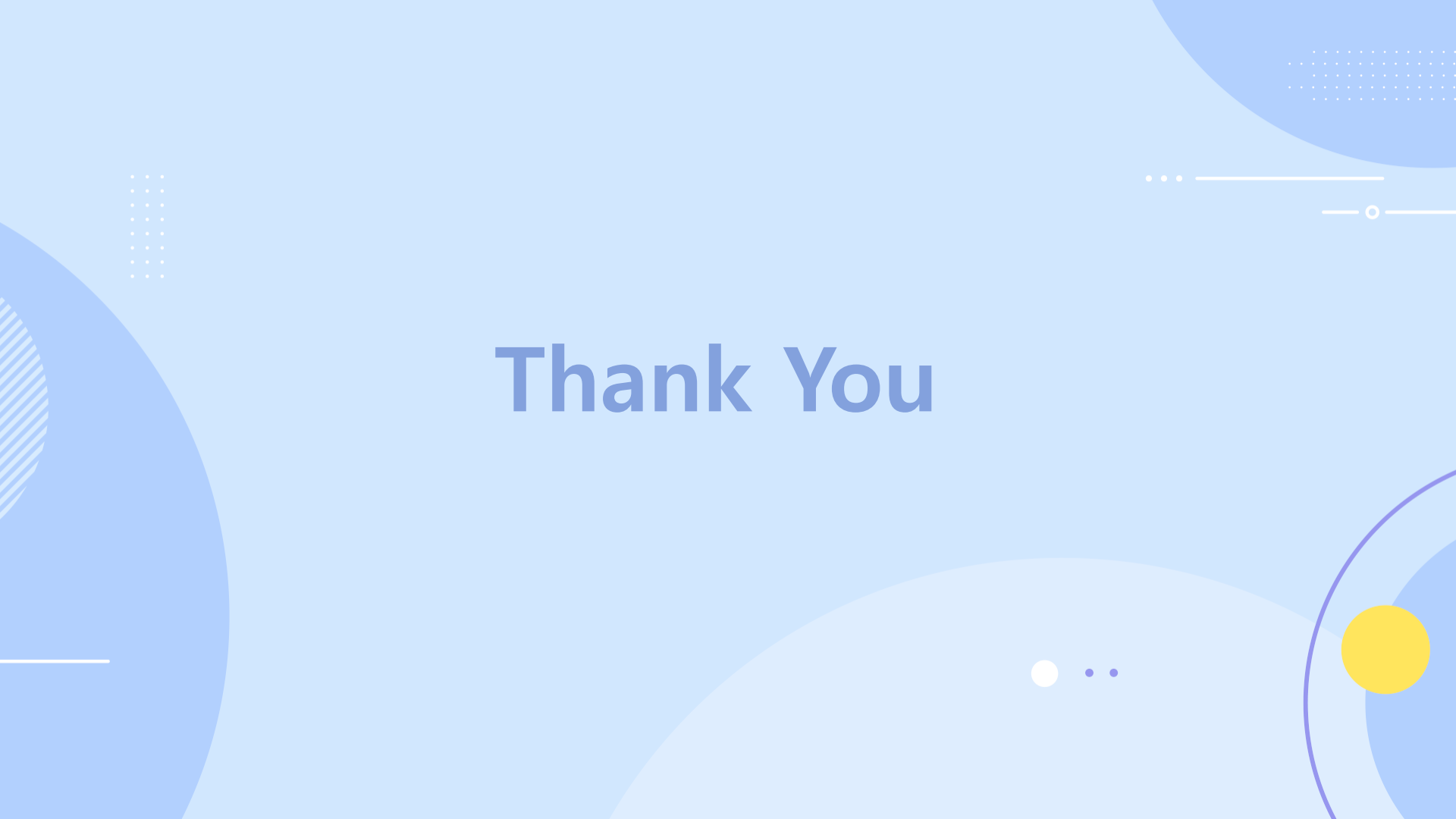
```
public class Car {  
    Engine engine;  
    public Car() {  
        // initialize car  
    }  
    public void start(Key key) {  
        Doors doors = new Doors();  
        boolean authorized = key.turns();  
        if (authorized) {  
            engine.start();  
            updateDashboardDisplay();  
        }  
    }  
    public void updateDashBoardDisplay() {  
        //  
    }  
}
```

Review

- Provides a unified interface to a set of interfaces in a subsystem.
- Facade defines a higher-level interface that makes the subsystem easier to use

Review - Related Patterns

- Mediator
 - Mediator's colleagues are aware of Mediator
- Facade
 - Unidirectional rather than cooperative interactions between object and subsystem
 - The subsystem doesn't know about the Facade
 - Facade doesn't add functionality, Mediator does



Thank You